CARNET DE TRADING

Code Source Complet

MetaTrader 5 — Forex · CCXT — Cryptomonnaies

Version 1.2 — Documentation technique intégrale

Ce document regroupe l'arborescence du projet, la configuration, tous les modules Python, les adaptateurs de courtier, le moteur de backtest, l'API FastAPI, l'interface web, les tests automatisés et les instructions d'installation. Le code source de chaque fichier est reproduit intégralement, sans omission.

Avertissement : Ce projet est une base de développement renforcée, non auditée et non certifiée pour la production. Les ordres réels restent volontairement bloqués. Aucun résultat de backtest ne garantit de gains.

Projet	carnet_trading_app
Version	1.2 (renforcée)
Date	23 octobre 2026
Stack	Python 3.11+ · FastAPI · SQLite · MetaTrader5 · CCXT
Licence / statut	Base de développement — non production

1. Présentation et arborescence du projet

Le projet « Carnet de Trading » est une application locale de journal de trading et de simulation, étendue avec un moteur de backtest, une gestion de risque, et des connecteurs vers MetaTrader 5 (Forex) et les plateformes crypto via CCXT. L'API web (FastAPI) et l'interface HTML/JS permettent de suivre les trades papier, les limites de risque et le journal d'événements chaîné. Les ordres réels sont volontairement bloqués (HTTP 503) tant que les contrôles de risque et de rapprochement ne sont pas complets et audités.

Arborescence des fichiers livrés

```
carnet_trading_app/
■■■ src_main.py                # Application FastAPI principale (API + auth + DB)
■■■ backtest_engine.py         # Moteur de backtesting OHLCV + coûts + robustesse
■■■ live_broker.py             # Couche simulation / courtier papier
■■■ mt5_adapter.py             # Connecteur MetaTrader 5 (Forex)
■■■ crypto_adapter.py          # Connecteur CCXT (cryptomonnaies)
■■■ execution_router.py        # Routeur unifié submit / reconcile
■■■ requirements.txt           # Dépendances Python
■■■ .env.example               # Modèle de variables d'environnement
■■■ README.md                  # Documentation principale
■■■ INTEGRATION_MT5_CRYPTO.md  # Addendum d'intégration MT5 / Crypto
■■■ static/
■   ■■■ index.html             # Interface web
■   ■■■ style.css              # Feuille de style
■   ■■■ app.js                 # Logique front-end
■■■ tests/
■   ■■■ test_backtest_engine.py
■   ■■■ test_adapters.py
■   ■■■ test_risk.py
■■■ docs/
    ■■■ Carnet_Trading_Architecture_Complete.pdf
    ■■■ Carnet_Trading_Addendum_MT5_Crypto.pdf
```

Le présent document reprend chaque fichier source dans l'ordre des sections ci-dessous, avec chemin, intitulé et code intégral.

2. Configuration et dépendances

Les dépendances sont listées dans `requirements.txt`. Python 3.11+ est recommandé. MetaTrader5 n'est installable que sous Windows (condition de plateforme dans le fichier). Les secrets d'authentification (JWT, Argon2id, TOTP) sont obligatoires au démarrage.

Fichier : `requirements.txt`

Dépendances Python du projet

Liste des packages nécessaires. MetaTrader5 est conditionnel Windows.

```
fastapi>=0.115,<1.0
uvicorn[standard]>=0.30,<1.0
argon2-cffi>=23.1
pyotp>=2.9
python-jose[cryptography]>=3.3
python-multipart>=0.0.9
pandas>=2.2
numpy>=2.0
ccxt>=4.4
pytest>=8.0
httpx>=0.27
# MetaTrader 5 Python bridge is Windows-only; install on the machine running the MT5 terminal.
MetaTrader5>=5.0.45; platform_system == "Windows"
```

Fichier : `.env.example`

Modèle de variables d'environnement

Copier vers .env et générer des secrets uniques. Ne jamais committer .env.

```
# Obligatoire avant démarrage — générer des valeurs uniques, ne jamais committer ce fichier.
JWT_SECRET=GENERER_UN_SECRET_ALEATOIRE_DE_48_OCTETS_OU_PLUS
APP_PASSWORD_HASH=COLLER_ICI_UN_HASH_ARGON2ID
TOTP_SECRET=GENERER_UN_SECRET_TOTP_BASE32
DATABASE_PATH=./carnet.db
REAL_TRADING_ENABLED=false
REAL_TRADING_REVIEWED=false
LIVE_ORDER_CONFIRMATION=
EXCHANGE_ID=
EXCHANGE_API_KEY=
EXCHANGE_API_SECRET=
EXCHANGE_PASSWORD=
EXCHANGE_MARKET_TYPE=spot
EXCHANGE_SANDBOX=true

# MetaTrader 5 Forex adapter (terminal must be installed and logged in / reachable)
MT5_TERMINAL_PATH=
MT5_LOGIN=
MT5_PASSWORD=
MT5_SERVER=
MT5_LIVE_ENABLED=false
MT5_ALLOW_REAL_ACCOUNT=false
MT5_MAX_DEVIATION_POINTS=20

# Crypto adapter (CCXT); sandbox remains mandatory by default
CRYPTO_EXCHANGE_ID=
CRYPTO_API_KEY=
CRYPTO_API_SECRET=
CRYPTO_SANDBOX=true
CRYPTO_LIVE_ENABLED=false
```

3. Chargement et préparation des données

Le chargement des données de marché (OHLCV) est principalement géré dans le moteur de backtest (`backtest_engine.py`) : validation des colonnes, horodatages, doublons, chronologie, gaps. L'application principale utilise SQLite pour les trades, les paramètres de risque et le journal d'événements.

Les fonctions de validation et de préparation se trouvent dans les modules suivants (code complet fourni dans les sections correspondantes).

4. Indicateurs et génération des signaux

Le moteur de backtest calcule les stops basés sur ATR, le sizing par risque, et applique les règles de sortie (stop / target). Les signaux d'entrée sont fournis sous forme de série (long/short/flat). Le code détaillé figure dans `backtest_engine.py` (section 5).

5. Moteur de backtesting

Module autonome de simulation historique : validation OHLCV, frais, spread, slippage, sizing par risque, stops ATR, coûts, découpage train/validation/test chronologique et analyses de robustesse de paramètres.

Fichier : `backtest_engine.py`

Moteur de backtest complet (OHLCV, coûts, robustesse)

```
"""Backtest event-driven with OHLC validation, fees, slippage, chronological holdout and robustness diagn
    ostics.
Research tool only: a passing test suite does not prove profitability or certify production readiness.
"""
from __future__ import annotations
import math
from dataclasses import dataclass, asdict
from typing import Iterable
import numpy as np
import pandas as pd

REQUIRED = ("open", "high", "low", "close", "volume")

@dataclass
class BacktestConfig:
    initial_capital: float = 10_000.0
    risk_fraction: float = 0.005
    fee_rate: float = 0.001
    slippage_rate: float = 0.0005
    spread_rate: float = 0.0002
    stop_atr_multiple: float = 2.0
    reward_risk: float = 2.0
    fast_window: int = 10
    slow_window: int = 30
    train_fraction: float = 0.60
    validation_fraction: float = 0.20
    max_leverage: float = 1.0

def validate_ohlcv(df: pd.DataFrame, max_gap_multiple: float = 3.0) -> pd.DataFrame:
    """Rejects malformed OHLCV, duplicate/out-of-order timestamps, invalid prices and bad volumes."""
    if not isinstance(df, pd.DataFrame) or len(df) < 100:
        raise ValueError("Au moins 100 bougies sont requises pour un test de recherche.")
    frame=df.copy()
    frame.columns=[str(c).strip().lower() for c in frame.columns]
    missing=set(REQUIRED)-set(frame.columns)
    if missing: raise ValueError(f"Colonnes manquantes: {sorted(missing)}")
    if "timestamp" not in frame.columns:
        raise ValueError("La colonne timestamp est obligatoire pour contrôler la chronologie.")
    frame["timestamp"]=pd.to_datetime(frame["timestamp"], utc=True, errors="coerce")
    if frame["timestamp"].isna().any(): raise ValueError("Timestamp invalide.")
    frame=frame.sort_values("timestamp",kind="stable").reset_index(drop=True)
    if frame["timestamp"].duplicated().any(): raise ValueError("Timestamps dupliqués.")
    # Reject original non-chronological input rather than silently repairing it.
    original=pd.to_datetime(df["timestamp"],utc=True,errors="coerce")
    if not original.is_monotonic_increasing: raise ValueError("Les données doivent déjà être triées par t
    imestamp croissant.")
    for col in ("open","high","low","close","volume"):
        frame[col]=pd.to_numeric(frame[col],errors="coerce")
        if frame[col].isna().any() or not np.isfinite(frame[col].to_numpy()).all():
            raise ValueError(f"Valeurs nulles ou non finies dans {col}.")
    if (frame[["open","high","low","close"]]<=0).any().any(): raise ValueError("Les prix doivent être str
    ictement positifs.")
    if (frame["volume"]<0).any(): raise ValueError("Le volume ne peut pas être négatif.")
    if (frame["high"] < frame[["open","close","low"]].max(axis=1)).any(): raise ValueError("Incohérence O
    HLC: high trop bas.")

    if (frame["low"] > frame[["open","close","high"]].min(axis=1)).any(): raise ValueError("Incohérence O
    HLC: low trop haut.")
    deltas=frame["timestamp"].diff().dropna()
    if len(deltas) and (deltas<=pd.Timedelta(0)).any(): raise ValueError("Chronologie invalide.")
    # Data gaps are reported and rejected by default to prevent accidental fabricated bars.
    if len(deltas):
        typical=deltas.median()
        if typical>pd.Timedelta(0) and (deltas>typical*max_gap_multiple).any():
            raise ValueError("Trous temporels anormaux détectés. Corrigez ou documentez les données avant
     backtest.")
    return frame

def _atr(frame: pd.DataFrame, window: int=14) -> pd.Series:
    prev=frame["close"].shift(1)
    tr=pd.concat([frame["high"]-frame["low"],(frame["high"]-prev).abs(),(frame["low"]-prev).abs()],axis=1
    ).max(axis=1)
    return tr.rolling(window,min_periods=window).mean()

def sma_cross_signals(frame: pd.DataFrame, fast: int, slow: int) -> pd.Series:
    """Signals use only fully closed bars; execution is at the next bar open."""
    if fast<2 or slow<=fast: raise ValueError("Paramètres SMA invalides: slow doit être supérieur à fast
    >= 2.")
    close=frame["close"]
    f=close.rolling(fast,min_periods=fast).mean()
```

```
    s=close.rolling(slow,min_periods=slow).mean()
    prev_gap=(f-s).shift(1)
    gap=f-s
    signal=pd.Series(0,index=frame.index,dtype=int)
    signal[(gap>0)&(prev_gap<=0)]=1
    signal[(gap<0)&(prev_gap>=0)]=-1
    return signal.fillna(0).astype(int)

def _simulate(frame: pd.DataFrame, cfg: BacktestConfig) -> dict:
    frame=frame.reset_index(drop=True).copy()
    signals=sma_cross_signals(frame,cfg.fast_window,cfg.slow_window)
    atr=_atr(frame)
    equity=cfg.initial_capital
    peak=equity
    max_dd=0.0
    pos=None
    trades=[]
    equity_curve=[]
    # signal generated at close i-1 can enter no earlier than open i.
    for i in range(1,len(frame)):
        bar=frame.iloc[i]
        # Stops/targets are evaluated before any new entry. If both touched, conservative stop-first.
        if pos:
            exit_price=None; reason=None
            if pos["side"]==1:
                if bar.low<=pos["stop"]: exit_price=pos["stop"]; reason="stop"
                elif bar.high>=pos["target"]: exit_price=pos["target"]; reason="target"
            else:
                if bar.high>=pos["stop"]: exit_price=pos["stop"]; reason="stop"
                elif bar.low<=pos["target"]: exit_price=pos["target"]; reason="target"
            if exit_price is not None:

                # adverse slippage on exit and fees on notional at entry+exit
                exit_exec=exit_price*(1-cfg.slippage_rate if pos["side"]==1 else 1+cfg.slippage_rate)
                gross=(exit_exec-pos["entry_exec"])*pos["qty"]*pos["side"]
                fees=cfg.fee_rate*pos["qty"]*(pos["entry_exec"]+exit_exec)
                net=gross-fees
                equity+=net
                trades.append({"entry_time":pos["entry_time"],"exit_time":bar.timestamp.isoformat(),
                    "side":"long" if pos["side"]==1 else "short","entry":pos["entry_exec"],
                    "exit":exit_exec,"quantity":pos["qty"],"gross_pnl":gross,"fees":fees,"pnl":net,
                    "R":net/pos["risk_cash"] if pos["risk_cash"] else None,"exit_reason":reason})
                pos=None
        # Use previous candle's signal and ATR; no current-bar close leakage.
        sig=int(signals.iloc[i-1])
        prev_atr=float(atr.iloc[i-1]) if pd.notna(atr.iloc[i-1]) else math.nan
        if pos is None and sig and math.isfinite(prev_atr) and prev_atr>0 and equity>0:
            raw_entry=float(bar.open)
            side=sig
            entry_exec=raw_entry*(1+cfg.slippage_rate+cfg.spread_rate/2 if side==1 else 1-cfg.slippage_ra
    te-cfg.spread_rate/2)
            stop_dist=cfg.stop_atr_multiple*prev_atr
            stop=entry_exec-stop_dist if side==1 else entry_exec+stop_dist
            target=entry_exec+cfg.reward_risk*stop_dist if side==1 else entry_exec-cfg.reward_risk*stop_d
    ist
            risk_cash=equity*cfg.risk_fraction
            qty=risk_cash/stop_dist
            notional=qty*entry_exec
            if qty>0 and notional<=equity*cfg.max_leverage:
                pos={"side":side,"entry":entry_exec,"entry_exec":entry_exec,"entry_time":bar.timestamp.is
    oformat(),
                     "qty":qty,"stop":stop,"target":target,"risk_cash":risk_cash}
        peak=max(peak,equity)
        max_dd=max(max_dd,(peak-equity)/peak if peak else 0)
        equity_curve.append({"timestamp":bar.timestamp.isoformat(),"equity":equity})
    # Close remaining position at final close, explicitly marked as forced close.
    if pos:
        last=frame.iloc[-1]; exit_exec=float(last.close)*(1-cfg.slippage_rate if pos["side"]==1 else 1+cf
    g.slippage_rate)
        gross=(exit_exec-pos["entry_exec"])*pos["qty"]*pos["side"]
        fees=cfg.fee_rate*pos["qty"]*(pos["entry_exec"]+exit_exec); net=gross-fees; equity+=net
        trades.append({"entry_time":pos["entry_time"],"exit_time":last.timestamp.isoformat(),
          "side":"long" if pos["side"]==1 else "short","entry":pos["entry_exec"],"exit":exit_exec,
          "quantity":pos["qty"],"gross_pnl":gross,"fees":fees,"pnl":net,
          "R":net/pos["risk_cash"] if pos["risk_cash"] else None,"exit_reason":"end_of_data"})
    return {"initial_capital":cfg.initial_capital,"final_equity":equity,"net_pnl":equity-cfg.initial_capi
    tal,
      "return_pct":(equity/cfg.initial_capital-1)*100,"max_drawdown_pct":max_dd*100,
      "trades":trades,"equity_curve":equity_curve}

def metrics(result: dict) -> dict:
    trades=result["trades"]; pnls=np.array([t["pnl"] for t in trades],dtype=float)
    wins=pnls[pnls>0]; losses=pnls[pnls<0]
    gross_win=float(wins.sum()); gross_loss=float(abs(losses.sum()))
    rvals=[t["R"] for t in trades if t["R"] is not None and math.isfinite(t["R"])]
    # Daily equity returns approximated from closed trades: not a Sharpe ratio unless daily sampling is u
    sed.

    return {"initial_capital":round(result["initial_capital"],2),"final_equity":round(result["final_equit
    y"],2),
```

```
      "net_pnl":round(result["net_pnl"],2),"return_pct":round(result["return_pct"],4),
      "trade_count":len(trades),"win_rate_pct":round(100*len(wins)/len(trades),3) if trades else 0,
      "profit_factor":round(gross_win/gross_loss,4) if gross_loss>0 else None,
      "expectancy_per_trade":round(float(pnls.mean()),4) if len(pnls) else None,
      "expectancy_R":round(float(np.mean(rvals)),4) if rvals else None,
      "max_drawdown_pct":round(result["max_drawdown_pct"],4),
      "gross_wins":round(gross_win,2),"gross_losses":round(gross_loss,2),
      "fees_total":round(sum(t["fees"] for t in trades),2),
      "warning":"Résultats historiques, sensibles à la qualité des données et aux hypothèses de frais/sli
    ppage; ne prédisent pas les résultats futurs."}

def run_validation(raw: pd.DataFrame, cfg: BacktestConfig|None=None) -> dict:
    cfg=cfg or BacktestConfig()
    if cfg.initial_capital<=0 or not 0<cfg.risk_fraction<=0.02: raise ValueError("Capital ou risque inval
    ide.")
    if min(cfg.fee_rate,cfg.slippage_rate,cfg.spread_rate)<0: raise ValueError("Frais, spread et slippage
     doivent être >= 0.")
    if cfg.max_leverage<=0 or cfg.max_leverage>2: raise ValueError("Levier max doit être >0 et <=2.")
    if cfg.fast_window<2 or cfg.slow_window<=cfg.fast_window: raise ValueError("Fenêtres SMA invalides.")
    frame=validate_ohlcv(raw)
    n=len(frame); train_end=int(n*cfg.train_fraction); val_end=int(n*(cfg.train_fraction+cfg.validation_f
    raction))
    if not (cfg.slow_window+20 < train_end < val_end < n):
        raise ValueError("Données insuffisantes pour train/validation/test chronologiques.")
    # Strict chronological splits. Parameters are fixed here; no tuning on test data.
    train=frame.iloc[:train_end].copy()
    validation=frame.iloc[train_end:val_end].copy()
    test=frame.iloc[val_end:].copy()
    # Warm-up history is used only for indicators; scored period starts at the split boundary.
    def segment(segment_frame, history):
        combined=pd.concat([history.tail(cfg.slow_window+20),segment_frame],ignore_index=True)
        res=_simulate(combined,cfg)
        # Since trades can originate in warm-up, report only those whose entry is in the scored segment.
        start=segment_frame.iloc[0].timestamp
        res["trades"]=[t for t in res["trades"] if pd.Timestamp(t["entry_time"])>=start]
        # Recompute score from retained trades rather than falsely claim continuity accounting.
        pnl=sum(t["pnl"] for t in res["trades"])
        res["net_pnl"]=pnl; res["final_equity"]=cfg.initial_capital+pnl
        res["return_pct"]=(res["final_equity"]/cfg.initial_capital-1)*100
        return metrics(res)
    train_result=segment(train,train.iloc[:0])
    val_result=segment(validation,train)
    test_result=segment(test,pd.concat([train,validation]))
    # Robustness grid: predeclared nearby parameters, all costs kept in place; diagnostic not a guarantee
    .
    robustness=[]
    for fast,slow in [(max(2,cfg.fast_window-2),cfg.slow_window),
                      (cfg.fast_window,cfg.slow_window+5),
                      (cfg.fast_window+2,cfg.slow_window+10)]:
        if fast>=slow: continue
        alt=BacktestConfig(**{**asdict(cfg),"fast_window":fast,"slow_window":slow})
        rr=_simulate(frame,alt)
        robustness.append({"fast":fast,"slow":slow,**metrics(rr)})

    return {"data":{"bars":n,"start":frame.timestamp.iloc[0].isoformat(),"end":frame.timestamp.iloc[-1].i
    soformat(),
                    "chronological":True,"data_validation":"passed"},
      "assumptions":{"fees_rate":cfg.fee_rate,"slippage_rate":cfg.slippage_rate,"spread_rate":cfg.spread_
    rate,
          "risk_fraction":cfg.risk_fraction,"max_leverage":cfg.max_leverage,"stop":"ATR précédent × multi
    ple",
          "entry":"open de la bougie suivant le signal clôturé","intrabar_tie":"stop prioritaire si stop
    et cible touchés"},
      "train":train_result,"validation":val_result,"out_of_sample_test":test_result,
      "robustness_nearby_parameters":robustness,
      "limitations":["Le modèle n'inclut pas les funding rates, emprunt, liquidations, règles fiscales, l
    atence ou profondeur de carnet.",
          "Le traitement des positions aux frontières des segments et l'exécution réelle doivent être aud
    ités pour chaque marché.",
          "Ne pas sélectionner une stratégie sur la base du seul résultat test; exiger walk-forward répét
    é et paper trading prolongé."]}
```

6. Gestion du capital et des risques

Les limites de risque (capital alloué, risque par trade, perte max jour/semaine, arrêt d'urgence) sont stockées en base et appliquées côté API. Le module `live_broker.py` gère le courtier papier et le calcul des quantités. Les tests de risque sont dans `tests/test_risk.py`.

Fichier : `live_broker.py`

Courtier papier / simulation et calcul de position

```
"""CCXT live adapter. Live orders are impossible unless explicitly enabled and all guardrails pass.
Venue-specific stop-order semantics vary; this adapter deliberately supports limit entries only and does
not claim to place a linked protective stop. Do not use live mode until venue-specific protection is impl
    emented.
"""
from __future__ import annotations
import os
from decimal import Decimal
import ccxt

def create_exchange():
    exchange_id=os.getenv("EXCHANGE_ID","").strip()
    api_key=os.getenv("EXCHANGE_API_KEY","").strip()
    secret=os.getenv("EXCHANGE_API_SECRET","").strip()
    if not exchange_id or not api_key or not secret:
        raise RuntimeError("EXCHANGE_ID, EXCHANGE_API_KEY et EXCHANGE_API_SECRET sont obligatoires.")
    if not hasattr(ccxt,exchange_id):
        raise RuntimeError(f"Exchange CCXT non reconnu: {exchange_id}")
    cls=getattr(ccxt,exchange_id)
    opts={"apiKey":api_key,"secret":secret,"enableRateLimit":True,
          "timeout":15000,"options":{"defaultType":os.getenv("EXCHANGE_MARKET_TYPE","spot")}}
    if os.getenv("EXCHANGE_PASSWORD"): opts["password"]=os.environ["EXCHANGE_PASSWORD"]
    ex=cls(opts)
    if os.getenv("EXCHANGE_SANDBOX","true").lower()=="true":
        if not ex.has.get("setSandboxMode"): raise RuntimeError("Cette plateforme ne déclare pas le mode
    sandbox CCXT.")
        ex.set_sandbox_mode(True)
    return ex

def validate_live_switch():
    if os.getenv("LIVE_TRADING_ENABLED","false").lower()!="true":
        raise RuntimeError("Trading réel désactivé. Définir LIVE_TRADING_ENABLED=true uniquement après au
    dit.")
    if os.getenv("LIVE_ORDER_CONFIRMATION")!="I_ACCEPT_REAL_FINANCIAL_RISK":
        raise RuntimeError("LIVE_ORDER_CONFIRMATION incorrecte.")
    if os.getenv("EXCHANGE_SANDBOX","true").lower()!="false":
        raise RuntimeError("Le mode sandbox est actif; configurez et testez le bac à sable avant la produ
    ction.")
    if os.getenv("REAL_TRADING_REVIEWED")!="true":
        raise RuntimeError("Le déploiement réel exige REAL_TRADING_REVIEWED=true après revue humaine docu
    mentée.")

def place_limit_order(symbol: str, side: str, amount: float, price: float):
    """No blind retry: a network timeout can mean the exchange accepted the order."""
    validate_live_switch()
    if side not in ("buy","sell"): raise ValueError("side doit être buy ou sell")
    if not symbol or "/" not in symbol: raise ValueError("Symbole CCXT attendu, ex. BTC/USDT")
    if not (amount>0 and price>0): raise ValueError("Quantité et prix doivent être positifs")
    ex=create_exchange()
    try:
        ex.load_markets()
        market=ex.market(symbol)
        if market.get("active") is False: raise RuntimeError("Marché inactif.")
        amount=float(ex.amount_to_precision(symbol,amount))
        price=float(ex.price_to_precision(symbol,price))

        if amount<=0 or price<=0: raise ValueError("Quantité/prix arrondi à zéro.")
        limits=market.get("limits") or {}
        min_amount=(limits.get("amount") or {}).get("min")
        min_cost=(limits.get("cost") or {}).get("min")
        if min_amount and amount<float(min_amount): raise ValueError("Quantité sous le minimum du marché.
    ")
        if min_cost and amount*price<float(min_cost): raise ValueError("Valeur notionnelle sous le minimu
    m du marché.")
        # Limit order only. No automatic retry after ambiguous network failures.
        return ex.create_order(symbol,"limit",side,amount,price,{"clientOrderId":os.urandom(12).hex()})
    finally:
        try: ex.close()
        except Exception: pass
```

Fichier : `tests/test_risk.py`

Tests unitaires des garde-fous de risque

```
from fastapi.testclient import TestClient
import os, tempfile
```

```
os.environ["JWT_SECRET"]="x"*48
os.environ["APP_PASSWORD_HASH"]="$argon2id$v=19$m=65536,t=3,p=4$not-a-real-hash"
os.environ["TOTP_SECRET"]="JBSWY3DPEHPK3PXP"
from src_main import app, validate_limits, DEFAULT_LIMITS, ABSOLUTE
from fastapi import HTTPException
import pytest

def test_default_limits_inside_absolute_bounds():
    for key, value in DEFAULT_LIMITS.items():
        lo, hi = ABSOLUTE[key]
        assert lo <= value <= hi

def test_risk_limit_rejects_excess():
    bad = dict(DEFAULT_LIMITS)
    bad["risque_par_trade"] = 0.5
    with pytest.raises(HTTPException):
        validate_limits(bad)

def test_daily_trade_limit_cannot_exceed_absolute():
    bad = dict(DEFAULT_LIMITS)
    bad["trades_max_jour"] = 6
    with pytest.raises(HTTPException):
        validate_limits(bad)
```

7. Connecteur MetaTrader 5 — Forex

Adaptateur Python vers le terminal MetaTrader 5. Connexion, vérification symbole et volume, `order_check` avant envoi, SL/TP natifs inclus dans la requête, rapprochement via `positions_get`. Terminal Windows et compte (démo recommandé) requis. L'activation live est protégée par drapeaux d'environnement.

Fichier : `mt5_adapter.py`

Adaptateur MetaTrader 5 (Forex) — connexion, ordre, rapprochement

```
"""MetaTrader 5 adapter. Requires Windows + installed MT5 terminal + demo account testing.
Real trading is fail-closed and requires multiple explicit gates. Never auto-retry ambiguous orders.
"""
from __future__ import annotations
import os
from typing import Any

class BrokerError(RuntimeError):
    pass

class MetaTrader5Adapter:
    def __init__(self, mt5_module=None):
        if mt5_module is None:
            try:
                import MetaTrader5 as mt5_module
            except ImportError as exc:
                raise BrokerError("Installer MetaTrader5 sur Windows avec le terminal MT5 installé.") fro
    m exc
        self.mt5 = mt5_module

    @staticmethod
    def _live_gate():
        if os.getenv("LIVE_TRADING_ENABLED", "false").lower() != "true":
            raise BrokerError("Trading réel désactivé.")
        if os.getenv("LIVE_ORDER_CONFIRMATION") != "I_ACCEPT_REAL_FINANCIAL_RISK":
            raise BrokerError("Confirmation de risque absente.")
        if os.getenv("REAL_TRADING_REVIEWED") != "true":
            raise BrokerError("Revue humaine documentée requise.")
        if os.getenv("MT5_LIVE_ENABLED", "false").lower() != "true":
            raise BrokerError("MT5 réel désactivé par défaut.")

    def connect(self):
        kwargs = {}
        if os.getenv("MT5_TERMINAL_PATH"):
            kwargs["path"] = os.environ["MT5_TERMINAL_PATH"]
        login = os.getenv("MT5_LOGIN", "").strip()
        password = os.getenv("MT5_PASSWORD", "")
        server = os.getenv("MT5_SERVER", "").strip()
        if login:
            kwargs["login"] = int(login)
        if password:
            kwargs["password"] = password
        if server:
            kwargs["server"] = server
        if not self.mt5.initialize(**kwargs):
            raise BrokerError(f"Initialisation MT5 échouée: {self.mt5.last_error()}")
        account = self.mt5.account_info()
        if account is None:
            self.mt5.shutdown()
            raise BrokerError("Compte MT5 indisponible.")
        # Demo accounts only unless an additional explicit production gate is set.
        demo_mode = getattr(self.mt5, "ACCOUNT_TRADE_MODE_DEMO", 0)
        if getattr(account, "trade_mode", None) != demo_mode:
            if os.getenv("MT5_ALLOW_REAL_ACCOUNT", "false").lower() != "true":
                self.mt5.shutdown()

                raise BrokerError("Compte non démo : MT5_ALLOW_REAL_ACCOUNT=true est requis après revue i
    ndépendante.")
            self._live_gate()
        return account

    def close(self):
        self.mt5.shutdown()

    def place_market_order(self, symbol: str, side: str, volume: float, stop_loss: float, take_profit: fl
    oat | None = None, *, magic: int = 620261):
        """Market order with SL and optional TP included in the native MT5 request."""
        self._live_gate()
        if side not in ("buy", "sell") or not symbol or volume <= 0 or stop_loss <= 0:
            raise ValueError("Symbole, sens, volume et stop-loss valides requis.")
        account = self.connect()
        try:
            if not self.mt5.symbol_select(symbol, True):
                raise BrokerError(f"Symbole indisponible: {symbol}")
            info = self.mt5.symbol_info(symbol)
            tick = self.mt5.symbol_info_tick(symbol)
            if info is None or tick is None:
                raise BrokerError("Informations de marché MT5 indisponibles.")
```

```
        step = float(getattr(info, "volume_step", 0) or 0)
        minvol = float(getattr(info, "volume_min", 0) or 0)
        maxvol = float(getattr(info, "volume_max", 0) or 0)
        if step <= 0 or volume < minvol or volume > maxvol:
            raise ValueError("Volume hors limites MT5.")
        steps = round(volume / step)
        normalized = round(steps * step, 8)
        if abs(normalized - volume) > max(1e-8, step * 1e-6):
            raise ValueError("Volume non conforme au pas du symbole.")
        order_type = self.mt5.ORDER_TYPE_BUY if side == "buy" else self.mt5.ORDER_TYPE_SELL
        price = float(tick.ask if side == "buy" else tick.bid)
        digits = int(getattr(info, "digits", 5))
        sl, tp = round(float(stop_loss), digits), (round(float(take_profit), digits) if take_profit e
lse 0.0)
        if side == "buy" and (sl >= price or (tp and tp <= price)):
            raise ValueError("Pour un achat, SL < prix courant et TP > prix courant.")
        if side == "sell" and (sl <= price or (tp and tp >= price)):
            raise ValueError("Pour une vente, SL > prix courant et TP < prix courant.")
        filling = getattr(info, "filling_mode", getattr(self.mt5, "ORDER_FILLING_IOC", 1))
        request = {
            "action": self.mt5.TRADE_ACTION_DEAL, "symbol": symbol, "volume": normalized,
            "type": order_type, "price": price, "sl": sl, "tp": tp,
            "deviation": int(os.getenv("MT5_MAX_DEVIATION_POINTS", "20")),
            "magic": magic, "comment": "CarnetTrading", "type_time": self.mt5.ORDER_TIME_GTC,
            "type_filling": filling,
        }
        check = self.mt5.order_check(request)
        if check is None or getattr(check, "retcode", None) not in (0, getattr(self.mt5, "TRADE_RETCO
DE_DONE", 10009)):
            raise BrokerError(f"MT5 order_check refusé: {getattr(check, 'comment', None)}")
        result = self.mt5.order_send(request)
        accepted = {getattr(self.mt5, "TRADE_RETCODE_DONE", 10009), getattr(self.mt5, "TRADE_RETCODE_
DONE_PARTIAL", 10010)}

        if result is None or getattr(result, "retcode", None) not in accepted:
            raise BrokerError(f"MT5 ordre refusé ou résultat ambigu: {getattr(result, 'comment', None
)}; ne pas réessayer automatiquement.")
        # Reconcile with broker positions; require native SL to be visible before declaring protected
.
        positions = self.mt5.positions_get(symbol=symbol)
        matching = [p for p in (positions or []) if getattr(p, "magic", None) == magic and getattr(p,
 "comment", "") == "CarnetTrading"]
        if not matching or any(float(getattr(p, "sl", 0) or 0) <= 0 for p in matching):
            raise BrokerError("État non rapproché ou stop-loss absent côté courtier : intervention ma
nuelle immédiate requise; aucun retry.")
        return {"venue":"mt5","status":"accepted_protected","order":getattr(result,"order",None),"dea
l":getattr(result,"deal",None),"volume":normalized,"positions":[self._position_dict(p) for p in match
ing]}
    finally:
        self.close()

@staticmethod
def _position_dict(p):
    return {"ticket":getattr(p,"ticket",None),"symbol":getattr(p,"symbol",None),"type":getattr(p,"typ
e",None),"volume":getattr(p,"volume",None),"price_open":getattr(p,"price_open",None),"sl":getattr(p,"
sl",None),"tp":getattr(p,"tp",None),"profit":getattr(p,"profit",None),"magic":getattr(p,"magic",None)
}

def reconcile_positions(self):
    self.connect()
    try:
        positions = self.mt5.positions_get()
        if positions is None:
            raise BrokerError(f"Rapprochement MT5 impossible: {self.mt5.last_error()}")
        return [self._position_dict(p) for p in positions]
    finally:
        self.close()
```

8. Connecteur de plateforme crypto

Adaptateur CCXT indépendant. Sandbox activé par défaut. Vérification explicite du support stopLoss / takeProfit attachés. Aucun retry automatique après résultat ambigu. Lecture des soldes, ordres et positions lorsque la plateforme les expose.

Fichier : `crypto_adapter.py`

Adaptateur CCXT (cryptomonnaies) — sandbox, SL/TP, garde-fous

```
"""CCXT crypto adapter: only exchanges advertising attached SL/TP are eligible.
Sandbox is mandatory by default. Never retry order submissions after ambiguous network errors.
"""
from __future__ import annotations
import os

class CryptoBrokerError(RuntimeError):
    pass

class CryptoAdapter:
    def __init__(self, exchange=None):
        self.exchange = exchange

    @staticmethod
    def _live_gate():
        if os.getenv("LIVE_TRADING_ENABLED", "false").lower() != "true":
            raise CryptoBrokerError("Trading réel désactivé.")
        if os.getenv("LIVE_ORDER_CONFIRMATION") != "I_ACCEPT_REAL_FINANCIAL_RISK":
            raise CryptoBrokerError("Confirmation de risque absente.")
        if os.getenv("REAL_TRADING_REVIEWED") != "true":
            raise CryptoBrokerError("Revue humaine documentée requise.")
        if os.getenv("CRYPTO_LIVE_ENABLED", "false").lower() != "true":
            raise CryptoBrokerError("Trading crypto réel désactivé par défaut.")

    def _create_exchange(self):
        try:
            import ccxt
        except ImportError as exc:
            raise CryptoBrokerError("Installer ccxt pour utiliser le connecteur crypto.") from exc
        exchange_id = os.getenv("CRYPTO_EXCHANGE_ID", "").strip()
        key, secret = os.getenv("CRYPTO_API_KEY", "").strip(), os.getenv("CRYPTO_API_SECRET", "").strip()
        if not exchange_id or not key or not secret or not hasattr(ccxt, exchange_id):
            raise CryptoBrokerError("CRYPTO_EXCHANGE_ID, CRYPTO_API_KEY et CRYPTO_API_SECRET valides requ
    is.")
        cls = getattr(ccxt, exchange_id)
        ex = cls({"apiKey": key, "secret": secret, "enableRateLimit": True, "timeout": 15000})
        sandbox = os.getenv("CRYPTO_SANDBOX", "true").lower() == "true"
        if sandbox:
            if not ex.has.get("setSandboxMode"):
                raise CryptoBrokerError("Cette plateforme ne déclare pas de sandbox CCXT.")
            ex.set_sandbox_mode(True)
        return ex

    def place_market_order_with_protection(self, symbol: str, side: str, amount: float, stop_loss: float,
     take_profit: float | None = None):
        self._live_gate()
        if side not in ("buy", "sell") or not symbol or amount <= 0 or stop_loss <= 0:
            raise ValueError("Symbole, sens, quantité et stop-loss valides requis.")
        ex = self.exchange or self._create_exchange()
        owns = self.exchange is None
        try:
            ex.load_markets()
            market = ex.market(symbol)
            if market.get("active") is False:
                raise CryptoBrokerError("Marché inactif.")

            # CCXT explicitly warns that attached protective orders are exchange-specific.
            try:
                sl_supported = ex.feature_value(symbol, "createOrder", "stopLoss")
                tp_supported = (take_profit is None) or ex.feature_value(symbol, "createOrder", "takeProf
    it")
            except Exception:
                sl_supported, tp_supported = False, False
            if not sl_supported or not tp_supported:
                raise CryptoBrokerError("La plateforme ne confirme pas le support des SL/TP attachés via
    CCXT; ordre non envoyé.")
            amount = float(ex.amount_to_precision(symbol, amount))
            stop_loss = float(ex.price_to_precision(symbol, stop_loss))
            if amount <= 0:
                raise ValueError("Quantité arrondie à zéro.")
            params = {"stopLoss": {"triggerPrice": stop_loss}}
            if take_profit is not None:
                params["takeProfit"] = {"triggerPrice": float(ex.price_to_precision(symbol, take_profit))
    }
            # A single entry request with attached protective orders; never send unprotected entry.
            order = ex.create_order(symbol, "market", side, amount, None, params)
            return {"venue":"crypto","status":"submitted_with_attached_protection","order":order}
        except (CryptoBrokerError, ValueError):
```

```
        raise
    except Exception as exc:
        # Submission outcome may be ambiguous; caller must reconcile, never retry blindly.
        raise CryptoBrokerError(f"Résultat de soumission ambigu/échec: {type(exc).__name__}; rapproch
er ordres et soldes avant toute action.") from exc
    finally:
        if owns:
            try: ex.close()
            except Exception: pass

def reconcile(self, symbol: str | None = None):
    ex = self.exchange or self._create_exchange()
    owns = self.exchange is None
    try:
        ex.load_markets()
        result = {"venue":"crypto","open_orders":ex.fetch_open_orders(symbol) if ex.has.get("fetchOpe
nOrders") else None,
                  "balances":ex.fetch_balance()}
        if ex.has.get("fetchPositions"):
            result["positions"] = ex.fetch_positions([symbol] if symbol else None)
        else:
            result["positions"] = None  # Spot holdings are balances, not standardized positions.
        return result
    finally:
        if owns:
            try: ex.close()
            except Exception: pass
```

9. Routeur d'exécution des ordres

API Python unifiée : `submit(venue="mt5"|"crypto", ...)` et `reconcile(...)`. Les soldes et risques restent isolés par courtier ; aucune fusion automatique des capitaux.

Fichier : `execution_router.py`

Routeur unifié d'exécution et de rapprochement

```
"""Unified routing layer for Forex (MT5) and crypto (CCXT)."""
from mt5_adapter import MetaTrader5Adapter
from crypto_adapter import CryptoAdapter

class ExecutionRouter:
    def __init__(self, mt5_adapter=None, crypto_adapter=None):
        self.mt5 = mt5_adapter or MetaTrader5Adapter()
        self.crypto = crypto_adapter or CryptoAdapter()

    def submit(self, venue: str, **order):
        if venue == "mt5":
            return self.mt5.place_market_order(**order)
        if venue == "crypto":
            return self.crypto.place_market_order_with_protection(**order)
        raise ValueError("venue doit être 'mt5' ou 'crypto'.")

    def reconcile(self, venue: str, **kwargs):
        if venue == "mt5":
            return self.mt5.reconcile_positions()
        if venue == "crypto":
            return self.crypto.reconcile(**kwargs)
        raise ValueError("venue doit être 'mt5' ou 'crypto'.")
```

10. Base de données et API

Application FastAPI principale : schéma SQLite (settings, trades, events chaînés, pending_limits), endpoints d'authentification JWT + TOTP, gestion des trades papier, limites de risque, journal d'événements, interface statique. L'endpoint d'ordre réel renvoie volontairement HTTP 503.

Fichier : `src_main.py`

Application FastAPI — API, auth, SQLite, endpoints, risque

```
"""
Carnet de Trading — MVP local en simulation uniquement.
Les ordres réels sont volontairement absents. Ne pas exposer directement sur Internet.
"""
from __future__ import annotations
import os, sqlite3, secrets, hashlib, hmac, json, math
from datetime import datetime, timezone, date, timedelta
from pathlib import Path
from typing import Optional, Literal
from fastapi import FastAPI, HTTPException, Depends, Header, Request
from fastapi.responses import FileResponse, HTMLResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, Field, model_validator
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError
import pyotp
from jose import jwt, JWTError

BASE = Path(__file__).resolve().parent
DB_PATH = os.getenv("DATABASE_PATH", str(BASE / "carnet.db"))
JWT_SECRET = os.getenv("JWT_SECRET", "")
TOTP_SECRET = os.getenv("TOTP_SECRET", "")
REAL_TRADING_ENABLED = os.getenv("REAL_TRADING_ENABLED", "false").lower() == "true"
JWT_ALG = "HS256"
JWT_MINUTES = 30
ph = PasswordHasher()
app = FastAPI(title="Carnet de Trading", version="1.1.0")
app.mount("/static", StaticFiles(directory=BASE / "static"), name="static")

DEFAULT_LIMITS = {
    "capital_alloue": 500.0, "risque_par_trade": 0.005,
    "perte_max_jour": 0.02, "perte_max_semaine": 0.05,
    "drawdown_max": 0.10, "levier_max": 1.0,
    "pertes_consecutives_max": 3, "trades_max_jour": 3,
}
ABSOLUTE = {
    "capital_alloue": (100.0, 1_000_000_000.0),
    "risque_par_trade": (0.0001, 0.02), "perte_max_jour": (0.0, 0.05),
    "perte_max_semaine": (0.0, 0.10), "drawdown_max": (0.0, 0.20),
    "levier_max": (0.0, 2.0), "pertes_consecutives_max": (1, 4),
    "trades_max_jour": (1, 5),
}

def utcnow():
    return datetime.now(timezone.utc).isoformat()

def db():
    con = sqlite3.connect(DB_PATH, check_same_thread=False)
    con.row_factory = sqlite3.Row
    return con

def init_db():
    with db() as c:
        c.executescript("""
        CREATE TABLE IF NOT EXISTS settings (key TEXT PRIMARY KEY, value TEXT NOT NULL);

        CREATE TABLE IF NOT EXISTS trades (
          id INTEGER PRIMARY KEY AUTOINCREMENT, created_at TEXT NOT NULL,
          closed_at TEXT, instrument TEXT NOT NULL, side TEXT NOT NULL,
          entry REAL NOT NULL, stop REAL NOT NULL, target REAL,
          quantity REAL NOT NULL, status TEXT NOT NULL, exit_price REAL,
          pnl REAL NOT NULL DEFAULT 0, r_multiple REAL, note TEXT NOT NULL DEFAULT ''
        );
        CREATE TABLE IF NOT EXISTS events (
          id INTEGER PRIMARY KEY AUTOINCREMENT, created_at TEXT NOT NULL,
          event_type TEXT NOT NULL, details TEXT NOT NULL, previous_hash TEXT,
          event_hash TEXT NOT NULL
        );
        CREATE TABLE IF NOT EXISTS pending_limits (
          id INTEGER PRIMARY KEY CHECK(id=1), requested_at TEXT NOT NULL,
          apply_at TEXT NOT NULL, payload TEXT NOT NULL
        );
        """)
        for k,v in DEFAULT_LIMITS.items():
            c.execute("INSERT OR IGNORE INTO settings(key,value) VALUES(?,?)", (k,json.dumps(v)))
        c.execute("INSERT OR IGNORE INTO settings(key,value) VALUES('emergency_stop','false')")
        c.execute("INSERT OR IGNORE INTO settings(key,value) VALUES('peak_equity','500.0')")
        c.commit()
```

```
def log_event(kind, details):
    with db() as c:
        last = c.execute("SELECT event_hash FROM events ORDER BY id DESC LIMIT 1").fetchone()
        prev = last["event_hash"] if last else "GENESIS"
        body = json.dumps({"at":utcnow(),"type":kind,"details":details,"previous":prev}, sort_keys=True)
        digest = hashlib.sha256(body.encode()).hexdigest()
        c.execute("INSERT INTO events(created_at,event_type,details,previous_hash,event_hash) VALUES(?,?,
    ?,?,?)",
                  (utcnow(),kind,json.dumps(details),prev,digest))
        c.commit()

def get_setting(key):
    with db() as c:
        row = c.execute("SELECT value FROM settings WHERE key=?", (key,)).fetchone()
    return json.loads(row["value"]) if row else None

def set_setting(key, value):
    with db() as c:
        c.execute("INSERT INTO settings(key,value) VALUES(?,?) ON CONFLICT(key) DO UPDATE SET value=exclu
    ded.value",
                  (key,json.dumps(value)))
        c.commit()

def validate_limits(payload):
    out = {}
    for k,(lo,hi) in ABSOLUTE.items():
        if k not in payload: raise HTTPException(422, f"Limite manquante: {k}")
        val = payload[k]
        if isinstance(val,bool) or not isinstance(val,(int,float)) or not math.isfinite(val) or val < lo
    or val > hi:
            raise HTTPException(422, f"{k} doit être compris entre {lo} et {hi}")
        if k in ("pertes_consecutives_max","trades_max_jour") and int(val) != val:

            raise HTTPException(422, f"{k} doit être entier")
        out[k] = int(val) if k in ("pertes_consecutives_max","trades_max_jour") else float(val)
    if out["capital_alloue"] < 100: raise HTTPException(422, "Capital minimum: 100 USD")
    return out

def current_trades():
    with db() as c:
        return [dict(r) for r in c.execute("SELECT * FROM trades ORDER BY id DESC LIMIT 500")]

def closed_trades():
    return [t for t in current_trades() if t["status"] == "closed"]

def risk_state():
    limits = {k:get_setting(k) for k in DEFAULT_LIMITS}
    trades = closed_trades()
    now = datetime.now(timezone.utc)
    today = now.date()
    week_start = today - timedelta(days=today.weekday())
    day_pnl = week_pnl = 0.0
    losses_in_row = 0
    for t in trades:
        d = datetime.fromisoformat(t["closed_at"]).date()
        if d == today: day_pnl += t["pnl"]
        if d >= week_start: week_pnl += t["pnl"]
    for t in trades:
        if t["pnl"] < 0: losses_in_row += 1
        else: break
    capital = limits["capital_alloue"] + sum(t["pnl"] for t in trades)
    peak = max(float(get_setting("peak_equity") or capital), capital)
    set_setting("peak_equity", peak)
    dd = max(0.0, (peak-capital)/peak) if peak else 0.0
    return {"limits":limits, "capital":round(capital,2), "peak_equity":round(peak,2),
            "day_pnl":round(day_pnl,2), "week_pnl":round(week_pnl,2),
            "drawdown":round(dd,4), "losses_in_row":losses_in_row,
            "trades_today":sum(1 for t in trades if datetime.fromisoformat(t["closed_at"]).date()==today)
    }

def auth_required(authorization: Optional[str] = Header(default=None)):
    if not JWT_SECRET or len(JWT_SECRET) < 32:
        raise HTTPException(503, "Configuration requise: JWT_SECRET d'au moins 32 caractères.")
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(401, "Authentification requise")
    token = authorization[7:]
    try:
        payload = jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALG])
        if payload.get("sub") != "local-admin": raise JWTError("subject")
    except JWTError:
        raise HTTPException(401, "Jeton invalide ou expiré")
    return True

class LoginIn(BaseModel):
    password: str = Field(min_length=1, max_length=256)
    totp_code: str = Field(pattern=r"^\d{6}$")

class LimitsIn(BaseModel):

    capital_alloue: float
```

```
    risque_par_trade: float
    perte_max_jour: float
    perte_max_semaine: float
    drawdown_max: float
    levier_max: float
    pertes_consecutives_max: int
    trades_max_jour: int

class TradeIn(BaseModel):
    instrument: str = Field(min_length=1, max_length=30)
    side: Literal["long","short"]
    entry: float = Field(gt=0)
    stop: float = Field(gt=0)
    target: Optional[float] = Field(default=None, gt=0)
    quantity: float = Field(gt=0)
    note: str = Field(default="", max_length=500)

    @model_validator(mode="after")
    def prices_make_sense(self):
        if self.side == "long" and self.stop >= self.entry: raise ValueError("Pour un long, le stop doit
    être sous l'entrée.")
        if self.side == "short" and self.stop <= self.entry: raise ValueError("Pour un short, le stop doi
    t être au-dessus de l'entrée.")
        if self.target is not None:
            if self.side == "long" and self.target <= self.entry: raise ValueError("Cible long au-dessus
    de l'entrée requise.")
            if self.side == "short" and self.target >= self.entry: raise ValueError("Cible short sous l'e
    ntrée requise.")
        return self

class CloseIn(BaseModel):
    exit_price: float = Field(gt=0)

@app.on_event("startup")
def startup():
    # Fail closed: no app start without mandatory auth configuration.
    if not JWT_SECRET or len(JWT_SECRET) < 32:
        raise RuntimeError("JWT_SECRET absent ou trop court (32 caractères minimum).")
    if not os.getenv("APP_PASSWORD_HASH", "").startswith("$argon2"):
        raise RuntimeError("APP_PASSWORD_HASH Argon2id manquant ou invalide.")
    if not TOTP_SECRET:
        raise RuntimeError("TOTP_SECRET manquant.")
    try:
        pyotp.TOTP(TOTP_SECRET)
    except Exception as exc:
        raise RuntimeError("TOTP_SECRET invalide.") from exc
    init_db()

@app.get("/", response_class=HTMLResponse)
def index():
    return FileResponse(BASE / "static" / "index.html")

@app.get("/api/health")
def health():

    return {"status":"ok","mode":"simulation","real_trading_enabled":False}

@app.post("/api/connexion")
def login(data: LoginIn, request: Request):
    if not JWT_SECRET or len(JWT_SECRET) < 32: raise HTTPException(503,"JWT_SECRET non configuré.")
    password_hash = os.getenv("APP_PASSWORD_HASH","")
    if not password_hash: raise HTTPException(503,"APP_PASSWORD_HASH non configuré.")
    try:
        ph.verify(password_hash, data.password)
    except (VerifyMismatchError, Exception):
        log_event("login_failed", {"client": request.client.host if request.client else "unknown"})
        raise HTTPException(401,"Identifiants invalides")
    if not TOTP_SECRET or not pyotp.TOTP(TOTP_SECRET).verify(data.totp_code, valid_window=1):
        log_event("totp_failed", {"client": request.client.host if request.client else "unknown"})
        raise HTTPException(401,"Code 2FA invalide")
    token = jwt.encode({"sub":"local-admin","exp":datetime.now(timezone.utc)+timedelta(minutes=JWT_MINUTE
    S),
                        "jti":secrets.token_urlsafe(16)}, JWT_SECRET, algorithm=JWT_ALG)
    log_event("login_success", {})
    return {"access_token":token,"token_type":"bearer","expires_in":JWT_MINUTES*60}

@app.get("/api/etat", dependencies=[Depends(auth_required)])
def state():
    rs = risk_state()
    return {"mode":"simulation","arret_urgence":get_setting("emergency_stop"),
            "capital":rs["capital"],"etat_risque":rs,"transactions":current_trades(),
            "pending_limits":get_pending()}

def get_pending():
    with db() as c:
        r=c.execute("SELECT * FROM pending_limits WHERE id=1").fetchone()
    return dict(r) if r else None

@app.get("/api/transactions", dependencies=[Depends(auth_required)])
def transactions():
```

```
    return current_trades()

@app.put("/api/limites", dependencies=[Depends(auth_required)])
def update_limits(data: LimitsIn):
    new=validate_limits(data.model_dump())
    old={k:get_setting(k) for k in DEFAULT_LIMITS}
    if any(new[k] > old[k] for k in new):
        requested=datetime.now(timezone.utc)
        apply=requested+timedelta(hours=24)
        with db() as c:
            c.execute("INSERT INTO pending_limits(id,requested_at,apply_at,payload) VALUES(1,?,?,?) "
                      "ON CONFLICT(id) DO UPDATE SET requested_at=excluded.requested_at,apply_at=excluded
    .apply_at,payload=excluded.payload",
                      (requested.isoformat(),apply.isoformat(),json.dumps(new)))
            c.commit()
        log_event("limits_pending", {"old":old,"new":new,"apply_at":apply.isoformat()})
        return {"status":"pending","apply_at":apply.isoformat(),"message":"Toute hausse est différée de 2
    4 heures."}
    for k,v in new.items(): set_setting(k,v)
    log_event("limits_updated", {"old":old,"new":new,"effective":"immediate"})

    return {"status":"applied","limits":new}

def apply_pending_if_due():
    p=get_pending()
    if p and datetime.now(timezone.utc) >= datetime.fromisoformat(p["apply_at"]):
        new=validate_limits(json.loads(p["payload"]))
        for k,v in new.items(): set_setting(k,v)
        with db() as c: c.execute("DELETE FROM pending_limits WHERE id=1"); c.commit()
        log_event("limits_applied", {"limits":new})

@app.post("/api/arret-urgence", dependencies=[Depends(auth_required)])
def emergency_stop(confirmation: dict):
    if confirmation.get("confirmation") != "ARRETER": raise HTTPException(400,"Tapez exactement ARRETER."
    )
    set_setting("emergency_stop", True); log_event("emergency_stop",{})
    return {"status":"stopped"}

@app.post("/api/reactiver", dependencies=[Depends(auth_required)])
def reactivate(confirmation: dict):
    if confirmation.get("confirmation") != "REACTIVER": raise HTTPException(400,"Tapez exactement REACTIV
    ER.")
    set_setting("emergency_stop", False); log_event("reactivated",{})
    return {"status":"active"}

@app.post("/api/transactions", dependencies=[Depends(auth_required)])
def open_trade(data: TradeIn):
    apply_pending_if_due()
    if get_setting("emergency_stop"): raise HTTPException(423,"Arrêt d'urgence actif.")
    rs=risk_state(); lim=rs["limits"]
    if rs["trades_today"] >= lim["trades_max_jour"]: raise HTTPException(409,"Plafond de trades quotidien
    s atteint.")
    if rs["losses_in_row"] >= lim["pertes_consecutives_max"]: raise HTTPException(409,"Plafond de pertes
    consécutives atteint.")
    if rs["day_pnl"] <= -lim["capital_alloue"]*lim["perte_max_jour"]: raise HTTPException(409,"Limite de
    perte journalière atteinte.")
    if rs["week_pnl"] <= -lim["capital_alloue"]*lim["perte_max_semaine"]: raise HTTPException(409,"Limite
     de perte hebdomadaire atteinte.")
    if rs["drawdown"] >= lim["drawdown_max"]: raise HTTPException(409,"Drawdown maximal atteint.")
    if data.side=="long" and data.stop>=data.entry: raise HTTPException(422,"Stop long invalide.")
    risk=abs(data.entry-data.stop)*data.quantity
    max_risk=rs["capital"]*lim["risque_par_trade"]
    if risk > max_risk + 1e-9: raise HTTPException(409,f"Risque du trade ({risk:.2f}) supérieur au plafon
    d ({max_risk:.2f}).")
    exposure=data.entry*data.quantity
    if rs["capital"] and exposure/rs["capital"] > lim["levier_max"]+1e-9:
        raise HTTPException(409,"Levier/exposition supérieur à la limite.")
    with db() as c:
        cur=c.execute("INSERT INTO trades(created_at,instrument,side,entry,stop,target,quantity,status,no
    te) VALUES(?,?,?,?,?,?,?,'open',?)",
          (utcnow(),data.instrument.upper(),data.side,data.entry,data.stop,data.target,data.quantity,data
    .note))
        trade_id=cur.lastrowid; c.commit()
    log_event("paper_trade_opened", {"trade_id":trade_id,"risk":risk})
    return {"id":trade_id,"status":"open","risk_usd":round(risk,2),"mode":"simulation"}

@app.post("/api/transactions/{trade_id}/cloturer", dependencies=[Depends(auth_required)])
def close_trade(trade_id: int, data: CloseIn):
    with db() as c:
        row=c.execute("SELECT * FROM trades WHERE id=? AND status='open'",(trade_id,)).fetchone()
        if not row: raise HTTPException(404,"Trade ouvert introuvable.")
        t=dict(row)
        pnl=((data.exit_price-t["entry"]) if t["side"]=="long" else (t["entry"]-data.exit_price))*t["quan
    tity"]
        initial_risk=abs(t["entry"]-t["stop"])*t["quantity"]
        r=pnl/initial_risk if initial_risk else None
        c.execute("UPDATE trades SET status='closed',closed_at=?,exit_price=?,pnl=?,r_multiple=? WHERE id
    =?",
                  (utcnow(),data.exit_price,pnl,r,trade_id)); c.commit()
    log_event("paper_trade_closed", {"trade_id":trade_id,"exit_price":data.exit_price,"pnl":round(pnl,2)}
```

```
    )
    return {"id":trade_id,"pnl":round(pnl,2),"r_multiple":round(r,3) if r is not None else None}

@app.get("/api/evenements", dependencies=[Depends(auth_required)])
def events():
    with db() as c:
        rows=c.execute("SELECT id,created_at,event_type,details,previous_hash,event_hash FROM events ORDE
    R BY id DESC LIMIT 200").fetchall()
    return [dict(r) for r in rows]

@app.post("/api/backtest", dependencies=[Depends(auth_required)])
def backtest(payload: dict):
    """Validated historical backtest with chronological train/validation/OOS split."""
    import pandas as pd
    from backtest_engine import BacktestConfig, run_validation
    candles=payload.get("candles")
    if not isinstance(candles,list) or len(candles)<100:
        raise HTTPException(422,"Fournir au moins 100 bougies OHLCV avec timestamp.")
    try:
        frame=pd.DataFrame(candles)
        cfg=BacktestConfig(
            initial_capital=float(payload.get("initial_capital",10000)),
            risk_fraction=float(payload.get("risk_fraction",0.005)),
            fee_rate=float(payload.get("fee_rate",0.001)),
            slippage_rate=float(payload.get("slippage_rate",0.0005)),
            spread_rate=float(payload.get("spread_rate",0.0002)),
            stop_atr_multiple=float(payload.get("stop_atr_multiple",2.0)),
            reward_risk=float(payload.get("reward_risk",2.0)),
            fast_window=int(payload.get("fast_window",10)),
            slow_window=int(payload.get("slow_window",30)),
            train_fraction=float(payload.get("train_fraction",0.60)),
            validation_fraction=float(payload.get("validation_fraction",0.20)),
            max_leverage=float(payload.get("max_leverage",1.0)),
        )
        return run_validation(frame,cfg)
    except (ValueError, TypeError, KeyError) as exc:
        raise HTTPException(422,f"Backtest refusé : {exc}") from exc

@app.post("/api/live/order", dependencies=[Depends(auth_required)])
def live_order(payload: dict):
    """Explicitly gated CCXT limit order. Venue-specific protective stops are not yet certified."""

    # Safety gate: the generic CCXT adapter does not guarantee an exchange-native
    # protective stop linked to a filled entry. Never send a live order until a
    # platform-specific implementation and tests replace this guard.
    raise HTTPException(503, "Trading réel verrouillé : sélectionnez la plateforme et implémentez/testez
    son stop protecteur natif avant toute transmission.")
    from live_broker import place_limit_order
    try:
        symbol=str(payload.get("symbol",""))
        side=str(payload.get("side",""))
        amount=float(payload.get("amount",0))
        price=float(payload.get("price",0))
        # independent, mandatory risk cap; no leverage beyond configured account risk
        rs=risk_state()
        limits=rs["limits"]
        if get_setting("emergency_stop"): raise HTTPException(423,"Arrêt d'urgence actif.")
        if amount*price > rs["capital"]*limits["levier_max"]:
            raise HTTPException(409,"Valeur de l'ordre supérieure au plafond d'exposition.")
        if amount*abs(price-float(payload.get("stop_price",0))) > rs["capital"]*limits["risque_par_trade"
    ]:
            raise HTTPException(409,"Risque estimé supérieur au plafond. Fournir un stop_price valide.")
        if float(payload.get("stop_price",0)) <= 0: raise HTTPException(422,"stop_price obligatoire.")
        result=place_limit_order(symbol,side,amount,price)
        log_event("live_order_submitted", {"symbol":symbol,"side":side,"amount":amount,
                  "price":price,"exchange_order_id":result.get("id")})
        return {"status":"submitted","exchange_order_id":result.get("id"),
                "warning":"Ordre transmis; vérifiez l'état sur la plateforme. Le stop protecteur doit êtr
    e géré par un adaptateur propre à cette plateforme."}
    except HTTPException: raise
    except Exception as exc:
        log_event("live_order_error", {"error_type":type(exc).__name__})
        raise HTTPException(409, f"Ordre réel refusé : {type(exc).__name__}: {exc}") from exc

@app.get("/api/app.js")
def app_js():
    return FileResponse(BASE/"static"/"app.js", media_type="application/javascript")

@app.middleware("http")
async def security_headers(request: Request, call_next):
    response=await call_next(request)
    response.headers["Content-Security-Policy"]="default-src 'self'; script-src 'self'; style-src 'self';
     img-src 'self' data:; connect-src 'self'; frame-ancestors 'none'; base-uri 'self'; form-action 'self
    '"
    response.headers["X-Frame-Options"]="DENY"
    response.headers["X-Content-Type-Options"]="nosniff"
    response.headers["Referrer-Policy"]="strict-origin-when-cross-origin"
    response.headers["Permissions-Policy"]="camera=(), microphone=(), geolocation=()"
    return response
```

Interface web (static)

Fichier : `static/index.html`

Page HTML de l'interface utilisateur

```
<!doctype html>
<html lang="fr">
<head>
<meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1,viewport-fit=cove
    r">
<meta name="theme-color" content="#0e1014"><title>Carnet de trading</title>
<link rel="stylesheet" href="/static/style.css"><script src="/api/app.js" defer></script>
</head>
<body>
<header><div><span class="eyebrow">JOURNAL DE MARCHÉ</span><h1>Carnet de trading</h1></div><span class="p
    ill">SIMULATION UNIQUEMENT</span></header>
<main>
<section id="login-panel" class="panel login">
<h2>Connexion sécurisée</h2><p>Mot de passe et code 2FA à six chiffres.</p>
<form id="login-form"><label>Mot de passe<input id="password" type="password" autocomplete="current-passw
    ord" required></label>
<label>Code 2FA<input id="totp" inputmode="numeric" pattern="[0-9]{6}" maxlength="6" required></label><bu
    tton>Se connecter</button></form><p id="login-error" role="alert"></p>
</section>
<section id="dashboard" hidden>
<div class="toolbar"><p id="status" aria-live="polite">Chargement…</p><button id="logout" class="secondar
    y">Déconnexion</button></div>
<div class="grid">
<article class="panel hero"><span class="eyebrow">CAPITAL SIMULÉ</span><div id="capital" class="big">$500
    .00</div><p>Mode papier — aucun ordre réel</p></article>
<article class="panel"><h2>Budget de perte</h2><div id="risk-summary"></div></article>
</div>
<article class="panel"><h2>Ouvrir un trade papier</h2><form id="trade-form" class="form-grid">
<label>Instrument<input id="instrument" value="BTCUSDT" maxlength="30" required></label>
<label>Sens<select id="side"><option value="long">Long</option><option value="short">Short</option></sele
    ct></label>
<label>Entrée<input id="entry" type="number" min="0.000001" step="any" required></label>
<label>Stop-loss<input id="stop" type="number" min="0.000001" step="any" required></label>
<label>Take-profit facultatif<input id="target" type="number" min="0.000001" step="any"></label>
<label>Quantité<input id="quantity" type="number" min="0.000001" step="any" required></label>
<label class="wide">Note<input id="note" maxlength="500"></label><button class="wide">Valider selon les l
    imites</button></form><p id="trade-message" role="alert"></p></article>
<article class="panel"><h2>Transactions</h2><div class="table-wrap"><table><thead><tr><th>ID</th><th>Inst
    rument</th><th>Sens</th><th>Entrée</th><th>Stop</th><th>Quantité</th><th>Statut</th><th>PnL</th><th>A
    ction</th></tr></thead><tbody id="trades"></tbody></table></div></article>
<article class="panel"><h2>Arrêt d'urgence</h2><p>Arrête les nouvelles ouvertures de trades. Tapez exacte
    ment le mot demandé.</p>
<div class="form-grid"><label>Confirmation<input id="stop-confirm" placeholder="ARRETER"></label><button
    id="stop-button" class="danger">Activer l'arrêt</button>
<label>Confirmation réactivation<input id="resume-confirm" placeholder="REACTIVER"></label><button id="re
    sume-button" class="secondary">Réactiver</button></div><p id="emergency-message" role="alert"></p></a
    rticle>
<article class="panel"><h2>Passage d'ordre réel — accès verrouillé</h2>
<p>Le connecteur CCXT est préparé, mais l'ordre réel ne doit être activé qu'après sélection de la platefo
    rme, tests sandbox, mise en place d'un stop protecteur spécifique à la plateforme et revue de sécurit
    é. Un ordre limite peut être exécuté sans stop associé : ne l'utilisez pas en production dans cet éta
    t.</p>
<form id="live-form" class="form-grid">
<label>Symbole CCXT<input id="live-symbol" placeholder="BTC/USDT" required></label>
<label>Sens<select id="live-side"><option value="buy">Achat</option><option value="sell">Vente</option></

    select></label>
<label>Quantité<input id="live-amount" type="number" min="0.00000001" step="any" required></label>
<label>Prix limite<input id="live-price" type="number" min="0.00000001" step="any" required></label>
<label>Prix stop requis par contrôle<input id="live-stop" type="number" min="0.00000001" step="any" requi
    red></label>
<label class="wide">Confirmation exacte (uniquement après revue humaine)<input id="live-confirm" placehol
    der="I_ACCEPT_REAL_FINANCIAL_RISK" required></label>
<button class="wide danger" type="submit">Soumettre un ordre réel (verrouillé par configuration serveur)<
    /button>
</form><p id="live-message" role="alert"></p></article>
<article class="panel"><h2>Backtest historique avec validations</h2><p>CSV requis : timestamp, open, high
    , low, close, volume. Minimum 100 bougies chronologiques, sans doublons ni trous anormaux. Le rapport
     sépare train, validation, test hors échantillon et robustesse de paramètres. Vérifiez les hypothèses
     de coûts et les limites du modèle.</p>
<label>Fichier CSV<input id="csv" type="file" accept=".csv,text/csv"></label><button id="run-backtest">La
    ncer le backtest</button><pre id="backtest-result" aria-live="polite"></pre></article>
</section>
</main><footer>Version MVP · Testez d'abord sur des données historiques · Aucun conseil financier</footer
    >
</body></html>
```

Fichier : `static/style.css`

Feuille de style CSS

```
@import url('https://fonts.googleapis.com/css2?family=DM+Mono:wght@400;500&family=Instrument+Serif&displa
```

```
    y=swap');
:root{--encre:#0e1014;--surface:#14171d;--papier:#ece6d6;--discret:#8c909a;--ligne:#272b33;--gain:#6cd0a4
    ;--perte:#f07f66;--alerte:#e8b85d;font-family:Arial, sans-serif;color:var(--papier);background:var(--
    encre)}
*{box-sizing:border-box}body{margin:0;min-height:100vh}header{display:flex;justify-content:space-between;
    align-items:center;gap:16px;padding:24px clamp(16px,4vw,56px);border-bottom:1px solid var(--ligne)}h1
    ,h2,p{margin-top:0}h1{font:40px 'Instrument Serif',Georgia,serif;margin-bottom:0}h2{font-size:18px;fo
    nt-weight:600}.eyebrow{font:11px 'DM Mono',monospace;letter-spacing:.14em;color:var(--discret)}.pill{
    border:1px solid var(--alerte);color:var(--alerte);padding:8px 10px;font:11px 'DM Mono',monospace;bor
    der-radius:3px}main{max-width:1200px;margin:auto;padding:28px 18px 60px}.panel{background:var(--surfa
    ce);border:1px solid var(--ligne);border-radius:10px;padding:22px;margin-bottom:18px}.login{max-width
    :420px;margin:7vh auto}.login p, .panel p{color:var(--discret);line-height:1.5}.grid{display:grid;gri
    d-template-columns:1fr 1fr;gap:18px}.hero .big{font:clamp(40px,6vw,64px) 'Instrument Serif',Georgia,s
    erif;margin:12px 0;color:var(--gain)}label{display:flex;flex-direction:column;gap:8px;font-size:13px;
    color:var(--discret);margin-bottom:14px}input,select{background:#0c0e12;border:1px solid #343943;bord
    er-radius:5px;color:var(--papier);padding:12px;font:14px Arial;width:100%;min-width:0}input:focus,sel
    ect:focus,button:focus-visible{outline:2px solid var(--alerte);outline-offset:2px}button{background:v
    ar(--papier);color:var(--encre);border:0;border-radius:5px;padding:12px 16px;font-weight:700;cursor:p
    ointer}button:hover{filter:brightness(.92)}.secondary{background:#292e37;color:var(--papier)}.danger{
    background:var(--perte);color:#21100c}.form-grid{display:grid;grid-template-columns:repeat(3,minmax(0
    ,1fr));gap:0 14px}.wide{grid-column:1/-1}.toolbar{display:flex;justify-content:space-between;align-it
    ems:center}.table-wrap{overflow:auto}table{border-collapse:collapse;width:100%;font-size:13px}th,td{t
    ext-align:left;border-bottom:1px solid var(--ligne);padding:12px 9px;white-space:nowrap}th{color:var(
    --discret);font-weight:500}td button{padding:7px 9px;font-size:12px}#login-error,#trade-message,#emer
    gency-message{color:var(--perte);min-height:1em}pre{white-space:pre-wrap;overflow-wrap:anywhere;color
    :var(--gain)}footer{text-align:center;color:var(--discret);padding:20px;font-size:12px}#risk-summary{
    line-height:1.9}.good{color:var(--gain)}.bad{color:var(--perte)}[hidden]{display:none!important}@medi
    a(max-width:820px){.grid{grid-template-columns:1fr}.form-grid{grid-template-columns:1fr 1fr}header{al
    ign-items:flex-start;flex-direction:column}h1{font-size:34px}}@media(max-width:480px){.form-grid{grid
    -template-columns:1fr}.wide{grid-column:auto}.panel{padding:16px}main{padding-inline:12px}}@media(pre
    fers-reduced-motion:reduce){*{scroll-behavior:auto!important;animation:none!important;transition:none
    !important}}
```

Fichier : `static/app.js`

Logique JavaScript front-end (API calls, UI)

```
let jeton = null;
const $ = id => document.getElementById(id);
async function api(path, options={}) {
  const headers = new Headers(options.headers || {});
  if (jeton) headers.set("Authorization", `Bearer ${jeton}`);
  if (options.body && !headers.has("Content-Type")) headers.set("Content-Type","application/json");
  let response;
  try { response = await fetch(path,{...options,headers,credentials:"omit"}); }
  catch(e) { throw new Error("Erreur réseau : vérifiez que le serveur est lancé."); }
  const data = await response.json().catch(()=>({detail:"Réponse serveur illisible"}));
  if (response.status===401 && path!=="/api/connexion") { logout(); throw new Error("Session expirée, rec
    onnectez-vous."); }
  if (!response.ok) throw new Error(data.detail || `Erreur HTTP ${response.status}`);
  return data;
}
function logout(){jeton=null;$("dashboard").hidden=true;$("login-panel").hidden=false;}
$("login-form").addEventListener("submit",async e=>{
 e.preventDefault();$("login-error").textContent="";
 try{const d=await api("/api/connexion",{method:"POST",body:JSON.stringify({password:$("password").value,
    totp_code:$("totp").value})});jeton=d.access_token;$("password").value="";$("totp").value="";$("login
    -panel").hidden=true;$("dashboard").hidden=false;await refresh();}
 catch(err){$("login-error").textContent=err.message;}
});
$("logout").addEventListener("click",logout);
async function refresh(){
 const s=await api("/api/etat");$("capital").textContent=`$${Number(s.capital).toFixed(2)}`;
 const r=s.etat_risque, l=r.limits;
 $("status").textContent=s.arret_urgence?"ARRÊT D'URGENCE ACTIF":"Système actif · Simulation";
 $("status").className=s.arret_urgence?"bad":"good";
 $("risk-summary").innerHTML=`Perte du jour : <b class="${r.day_pnl<0?'bad':'good'}">$${r.day_pnl.toFixed
    (2)}</b><br>Perte semaine : <b class="${r.week_pnl<0?'bad':'good'}">$${r.week_pnl.toFixed(2)}</b><br>
    Drawdown : ${(r.drawdown*100).toFixed(2)}% / ${(l.drawdown_max*100).toFixed(1)}%<br>Trades aujourd'hu
    i : ${r.trades_today} / ${l.trades_max_jour}<br>Pertes consécutives : ${r.losses_in_row} / ${l.pertes
    _consecutives_max}<br>Arrêt d'urgence : ${s.arret_urgence?'ACTIF':'inactif'}`;
 renderTrades(s.transactions);
}
function renderTrades(trades){
 const body=$("trades");body.replaceChildren();
 for(const t of trades){const tr=document.createElement("tr");
   [t.id,t.instrument,t.side,t.entry,t.stop,t.quantity,t.status,t.status==="closed"?Number(t.pnl).toFixed
    (2):"—"].forEach(v=>{const td=document.createElement("td");td.textContent=String(v);tr.appendChild(td
    );});
   const td=document.createElement("td");
   if(t.status==="open"){const b=document.createElement("button");b.textContent="Clôturer";b.addEventList
    ener("click",async()=>{const px=prompt("Prix de sortie simulé :");if(!px||!Number.isFinite(Number(px)
    )||Number(px)<=0)return;try{const out=await api(`/api/transactions/${t.id}/cloturer`,{method:"POST",b
    ody:JSON.stringify({exit_price:Number(px)})});alert(`PnL simulé : $${out.pnl}`);await refresh();}catc
    h(e){alert(e.message)}});td.appendChild(b);}else td.textContent="—";
   tr.appendChild(td);body.appendChild(tr);
 }
}
```

```
$("trade-form").addEventListener("submit",async e=>{
 e.preventDefault();$("trade-message").textContent="";
 const v={instrument:$("instrument").value,side:$("side").value,entry:Number($("entry").value),stop:Numbe
    r($("stop").value),target:$("target").value?Number($("target").value):null,quantity:Number($("quantit

    y").value),note:$("note").value};
 try{const d=await api("/api/transactions",{method:"POST",body:JSON.stringify(v)});$("trade-message").tex
    tContent=`Trade papier #${d.id} ouvert. Risque simulé : $${d.risk_usd}`;await refresh();}
 catch(err){$("trade-message").textContent=err.message;}
});
$("stop-button").addEventListener("click",async()=>{try{await api("/api/arret-urgence",{method:"POST",bod
    y:JSON.stringify({confirmation:$("stop-confirm").value})});$("emergency-message").textContent="Arrêt
    activé.";await refresh();}catch(e){$("emergency-message").textContent=e.message}});
$("resume-button").addEventListener("click",async()=>{try{await api("/api/reactiver",{method:"POST",body:
    JSON.stringify({confirmation:$("resume-confirm").value})});$("emergency-message").textContent="Systèm
    e réactivé en mode simulation.";await refresh();}catch(e){$("emergency-message").textContent=e.messag
    e}});
$("run-backtest").addEventListener("click",async()=>{
 const f=$("csv").files[0];if(!f){$("backtest-result").textContent="Choisissez un CSV.";return;}
 try{
  const text=await f.text();const lines=text.trim().split(/\r?\n/);
  const headers=lines.shift().split(",").map(x=>x.trim().toLowerCase());
  const required=["timestamp","open","high","low","close","volume"];
  const missing=required.filter(h=>!headers.includes(h));
  if(missing.length)throw new Error("Colonnes obligatoires absentes : "+missing.join(", "));
  const candles=lines.filter(Boolean).map(line=>{const a=line.split(",");const o={};headers.forEach((h,i)
    =>o[h]=["timestamp"].includes(h)?a[i].trim():Number(a[i]));return o;});
  const result=await api("/api/backtest",{method:"POST",body:JSON.stringify({candles})});
  $("backtest-result").textContent=JSON.stringify(result,null,2);
 }catch(e){$("backtest-result").textContent=e.message;}
});

$("live-form").addEventListener("submit",async e=>{
 e.preventDefault();$("live-message").textContent="";
 if($("live-confirm").value!=="I_ACCEPT_REAL_FINANCIAL_RISK"){ $("live-message").textContent="Confirmatio
    n exacte requise.";return; }
 const body={symbol:$("live-symbol").value,side:$("live-side").value,amount:Number($("live-amount").value
    ),price:Number($("live-price").value),stop_price:Number($("live-stop").value)};
 try{const d=await api("/api/live/order",{method:"POST",body:JSON.stringify(body)});$("live-message").tex
    tContent=JSON.stringify(d);}
 catch(err){$("live-message").textContent=err.message;}
});
```

11. Sécurité et authentification

Au démarrage, l'application exige : JWT_SECRET (long), APP_PASSWORD_HASH (Argon2id) et TOTP_SECRET. Les mots de passe sont vérifiés avec Argon2id ; les sessions sont des JWT HS256 de courte durée. Le journal d'événements est chaîné par hash SHA-256. Les secrets ne doivent jamais être commités. Les clés API exchange/MT5 restent côté serveur avec permissions minimales.

Génération des secrets (exemples) :

```
python -c "from argon2 import PasswordHasher; print(PasswordHasher().hash(input('Mot de passe: ')))"
python -c "import secrets; print(secrets.token_urlsafe(48))"
python -c "import pyotp; print(pyotp.random_base32())"
```

Les drapeaux REAL_TRADING_ENABLED, MT5_LIVE_ENABLED, CRYPTO_LIVE_ENABLED et MT5_ALLOW_REAL_ACCOUNT protègent l'activation des ordres réels. L'API web bloque volontairement l'endpoint /api/live/order (HTTP 503) tant que les contrôles de risque et de rapprochement ne sont pas complets.

12. Rapprochement des positions avec les courtiers

Chaque adaptateur expose une logique de rapprochement :

- **MT5** : après order_send, lecture via positions_get ; refus de déclarer l'ordre protégé si la position ou le SL côté courtier n'est pas visible.
- **Crypto (CCXT)** : lecture des ordres, trades, soldes et positions (si exposées) ; aucun retry automatique après timeout (état ambigu).
- **Routeur** : méthode reconcile(venue=...) pour un point d'entrée unique.

Les soldes et risques doivent rester isolés par courtier et devise. Une vue consolidée nécessite une conversion explicite avec taux horodaté.

13. Tests automatisés

Les tests utilisent des faux courtiers (mocks) et ne communiquent pas avec un serveur MT5 réel ni un testnet crypto. Ils vérifient les contrats, les garde-fous et l'absence de retry aveugle. Installer requirements.txt puis lancer :

```
PYTHONPATH=. pytest -q
```

Fichier : `tests/test_backtest_engine.py`

Tests du moteur de backtest

```
import pandas as pd
import numpy as np
import pytest
from backtest_engine import validate_ohlcv, sma_cross_signals, BacktestConfig, run_validation

def sample(n=500):
    ts=pd.date_range("2024-01-01",periods=n,freq="h",tz="UTC")
    close=100+np.cumsum(np.sin(np.arange(n)/8)*0.3+0.02)
    op=np.r_[close[0],close[:-1]]
    high=np.maximum(op,close)+0.3
    low=np.minimum(op,close)-0.3
    return pd.DataFrame({"timestamp":ts,"open":op,"high":high,"low":low,"close":close,"volume":np.ones(n)
    *10})

def test_valid_ohlcv():
    assert len(validate_ohlcv(sample()))==500

def test_duplicate_timestamp_rejected():
    d=sample(); d.loc[5,"timestamp"]=d.loc[4,"timestamp"]
    with pytest.raises(ValueError): validate_ohlcv(d)

def test_bad_ohlc_rejected():
    d=sample(); d.loc[10,"high"]=1
    with pytest.raises(ValueError): validate_ohlcv(d)

def test_negative_volume_rejected():
    d=sample(); d.loc[10,"volume"]=-1
    with pytest.raises(ValueError): validate_ohlcv(d)

def test_unsorted_timestamps_rejected():
    d=sample(); d.loc[10,"timestamp"],d.loc[11,"timestamp"]=d.loc[11,"timestamp"],d.loc[10,"timestamp"]
    with pytest.raises(ValueError): validate_ohlcv(d)

def test_signals_are_same_for_truncated_prefix():
    d=sample()
    a=sma_cross_signals(d,10,30)
    b=sma_cross_signals(d.iloc[:200],10,30)
    pd.testing.assert_series_equal(a.iloc[:200].reset_index(drop=True),b.reset_index(drop=True))

def test_walk_forward_report_sections():
    report=run_validation(sample(),BacktestConfig())
    assert report["data"]["data_validation"]=="passed"
    assert "out_of_sample_test" in report
    assert "robustness_nearby_parameters" in report
```

Fichier : `tests/test_adapters.py`

Tests des adaptateurs MT5 et Crypto (simulés)

```
import os
from types import SimpleNamespace
import pytest
from mt5_adapter import MetaTrader5Adapter, BrokerError
from crypto_adapter import CryptoAdapter, CryptoBrokerError

class FakeMT5:
    TRADE_ACTION_DEAL=1; ORDER_TYPE_BUY=0; ORDER_TYPE_SELL=1; ORDER_TIME_GTC=0
    ORDER_FILLING_IOC=1; TRADE_RETCODE_DONE=10009; TRADE_RETCODE_DONE_PARTIAL=10010
    ACCOUNT_TRADE_MODE_DEMO=0
    def __init__(self): self.request=None
    def initialize(self, **kwargs): return True
    def shutdown(self): pass
    def account_info(self): return SimpleNamespace(trade_mode=0)
    def symbol_select(self, symbol, enabled): return True
    def symbol_info(self, symbol): return SimpleNamespace(volume_step=0.01,volume_min=0.01,volume_max=100
    ,digits=5,filling_mode=1)
    def symbol_info_tick(self, symbol): return SimpleNamespace(ask=1.1000,bid=1.0998)
    def order_check(self, req): return SimpleNamespace(retcode=0,comment="ok")
    def order_send(self, req):
        self.request=req
        return SimpleNamespace(retcode=10009,order=123,deal=456,comment="done")
    def positions_get(self, **kwargs): return [SimpleNamespace(ticket=123,symbol="EURUSD",type=0,volume=0
    .1,price_open=1.1,sl=1.09,tp=1.12,profit=0,magic=620261,comment="CarnetTrading")]
    def last_error(self): return (0,"ok")

def set_live_env(monkeypatch, prefix=None):
```

```
    for k,v in {"LIVE_TRADING_ENABLED":"true","LIVE_ORDER_CONFIRMATION":"I_ACCEPT_REAL_FINANCIAL_RISK","R
    EAL_TRADING_REVIEWED":"true","MT5_LIVE_ENABLED":"true","CRYPTO_LIVE_ENABLED":"true"}.items(): monkeyp
    atch.setenv(k,v)

def test_mt5_requires_live_gates(monkeypatch):
    monkeypatch.setenv("LIVE_TRADING_ENABLED","false")
    with pytest.raises(BrokerError): MetaTrader5Adapter(FakeMT5()).place_market_order("EURUSD","buy",0.1,
    1.09)

def test_mt5_sends_native_sl_tp_and_reconciles(monkeypatch):
    set_live_env(monkeypatch)
    fake=FakeMT5(); result=MetaTrader5Adapter(fake).place_market_order("EURUSD","buy",0.1,1.09,1.12)
    assert fake.request["sl"] == 1.09 and fake.request["tp"] == 1.12
    assert result["status"] == "accepted_protected"

def test_mt5_rejects_bad_stop(monkeypatch):
    set_live_env(monkeypatch)
    with pytest.raises(ValueError): MetaTrader5Adapter(FakeMT5()).place_market_order("EURUSD","buy",0.1,1
    .11)

class FakeCrypto:
    has={"fetchOpenOrders":True,"fetchPositions":False}
    def __init__(self, supports=True): self.supports=supports; self.params=None
    def load_markets(self): pass
    def market(self,symbol): return {"active":True}
    def feature_value(self,symbol,method,feature): return self.supports
    def amount_to_precision(self,symbol,amount): return str(amount)
    def price_to_precision(self,symbol,price): return str(price)

    def create_order(self,*args): self.params=args[-1]; return {"id":"abc"}
    def close(self): pass
    def fetch_open_orders(self,symbol=None): return []
    def fetch_balance(self): return {"USDT":{"free":100}}

def test_crypto_requires_attached_stop_support(monkeypatch):
    set_live_env(monkeypatch)
    with pytest.raises(CryptoBrokerError): CryptoAdapter(FakeCrypto(False)).place_market_order_with_prote
    ction("BTC/USDT","buy",0.01,60000)

def test_crypto_uses_attached_protection(monkeypatch):
    set_live_env(monkeypatch)
    fake=FakeCrypto(True)
    out=CryptoAdapter(fake).place_market_order_with_protection("BTC/USDT","buy",0.01,60000,70000)
    assert out["status"] == "submitted_with_attached_protection"
    assert fake.params["stopLoss"]["triggerPrice"] == 60000
    assert fake.params["takeProfit"]["triggerPrice"] == 70000

def test_crypto_does_not_retry_or_fallback_unprotected(monkeypatch):
    set_live_env(monkeypatch)
    class Broken(FakeCrypto):
        def create_order(self,*args): raise TimeoutError("ambiguous")
    with pytest.raises(CryptoBrokerError, match="ambigu"):
        CryptoAdapter(Broken(True)).place_market_order_with_protection("BTC/USDT","buy",0.01,60000)
```

14. Installation et instructions de lancement

Environnement recommandé : Python 3.11+, système isolé (venv). Pour MT5 : machine Windows avec terminal MetaTrader 5 installé et connecté.

Étapes d'installation

```
# 1. Créer et activer un environnement virtuel
python -m venv .venv
# Windows :
.venv\Scripts\activate
# Linux / macOS :
source .venv/bin/activate

# 2. Installer les dépendances
pip install -r requirements.txt

# 3. Configurer les secrets (copier .env.example → .env)
cp .env.example .env
# Éditer .env : JWT_SECRET, APP_PASSWORD_HASH, TOTP_SECRET, DATABASE_PATH

# 4. Lancer l'application (simulation uniquement)
uvicorn src_main:app --host 127.0.0.1 --port 8000

# 5. Ouvrir http://127.0.0.1:8000 dans le navigateur

# 6. Lancer les tests
PYTHONPATH=. pytest -q
```

Documentation complémentaire livrée

- `README.md` — statut, fonctionnalités, attention ordres réels, installation, secrets, architecture.
- `INTEGRATION_MT5_CRYPTO.md` — addendum d'implémentation Forex MT5 et cryptomonnaies (règles, blocage API, tests, conformité).
- `docs/Carnet_Trading_Architecture_Complete.pdf` et `docs/Carnet_Trading_Addendum_MT5_Crypto.pdf` — documents d'architecture déjà générés.

Statut final

Ce projet **n'est pas audité, certifié ni prêt pour le trading réel**. L'implémentation peut contenir des défauts. Les stops ne garantissent pas le prix d'exécution. Une revue indépendante du code, de la sécurité, des données et des opérations est requise avant tout usage en conditions réelles.

Annexe A — README.md (texte intégral)

Fichier : `README.md`

Documentation principale du dépôt

```
# Carnet de Trading — version renforcée 1.2

## Statut honnête
Ce dépôt est une base de développement renforcée, **pas une application auditée ni certifiée pour la prod
    uction**. Les tests automatisés sont des contrôles techniques et ne remplacent pas un audit indépenda
    nt, un examen de sécurité, des essais de marché ni une validation juridique/réglementaire. Aucun résu
    ltat de backtest ne garantit des gains.

## Fonctionnalités
- FastAPI + SQLite, interface web responsive.
- Configuration d'authentification obligatoire au démarrage : JWT secret long, hash Argon2id et secret TO
    TP.
- Trades papier, clôture, limites de risque, arrêt d'urgence, journal d'événements à empreintes chaînées.
- Moteur de backtest séparé : validation OHLCV, horodatages, doublons, chronologie, gaps, frais, spread,
    slippage, sizing par risque, stops ATR, coûts, train/validation/test chronologiques et robustesse de
    paramètres.
- Adaptateur CCXT préparé pour des ordres limite réels, protégé par plusieurs drapeaux d'environnement.

## Attention : passage d'ordres réels
Le connecteur CCXT ne suffit pas à garantir une protection de position. Les ordres stop conditionnels et
    leur association à l'ordre d'entrée diffèrent selon les plateformes. **Ne pas activer la production a
    vant d'avoir choisi une plateforme précise et implémenté/testé son adaptateur de stops protecteurs, s
    a gestion des ordres partiellement exécutés, ses idempotency keys, la réconciliation des positions et
```

```
un kill switch testé.** L'endpoint d'ordre réel renvoie volontairement HTTP 503 tant que l'adaptateu
r spécifique avec stop protecteur n'a pas été développé et audité. Le formulaire n'envoie donc actuel
lement aucun ordre réel.

## Installation
Python 3.11+ recommandé :
```bash
python -m venv .venv
# Windows : .venv\\Scripts\\activate
# Linux/macOS : source .venv/bin/activate
pip install -r requirements.txt
```

Secrets obligatoires avant le démarrage
Copier `.env.example` vers `.env` et générer les secrets. Ne pas committer `.env`.
```bash
python -c "from argon2 import PasswordHasher; print(PasswordHasher().hash(input('Mot de passe: ')))"
python -c "import secrets; print(secrets.token_urlsafe(48))"
python -c "import pyotp; print(pyotp.random_base32())"
```
Définir ensuite `JWT_SECRET`, `APP_PASSWORD_HASH`, `TOTP_SECRET`, `DATABASE_PATH` dans l'environnement de
l'application. Le démarrage échoue volontairement si la configuration d'authentification manque.

Exemple Linux/macOS :
```bash
export JWT_SECRET='SECRET_ALEATOIRE_LONG'
export APP_PASSWORD_HASH='HASH_ARGON2ID'
export TOTP_SECRET='SECRET_TOTP_BASE32'
export REAL_TRADING_ENABLED=false
uvicorn src_main:app --host 127.0.0.1 --port 8000
```
Ouvrir `http://127.0.0.1:8000`. L'hôte `127.0.0.1` limite l'accès à la machine locale. Configurez l'appli

cation d'authentification TOTP avec le secret indiqué; le parcours d'enrôlement QR n'est pas encore i
nclus.

Trading réel : configuration bloquée par défaut
Avant même d'envisager un ordre réel, choisir la plateforme et le type de marché. Les paramètres attendus
par l'adaptateur sont `EXCHANGE_ID`, `EXCHANGE_API_KEY`, `EXCHANGE_API_SECRET`, `EXCHANGE_MARKET_TYP
E`, `EXCHANGE_SANDBOX`, `LIVE_TRADING_ENABLED`, `LIVE_ORDER_CONFIRMATION`, `REAL_TRADING_REVIEWED`. L
es clés API doivent avoir les permissions minimales nécessaires; **désactiver les retraits** et, si p
ossible, restreindre l'IP. Ne jamais mettre ces secrets dans le navigateur, dans le PDF ou dans Git.
Le connecteur exige `LIVE_TRADING_ENABLED=true`, `EXCHANGE_SANDBOX=false`, `LIVE_ORDER_CONFIRMATION=I_ACC
EPT_REAL_FINANCIAL_RISK` et `REAL_TRADING_REVIEWED=true`. Ces drapeaux ne constituent pas un audit et
ne doivent pas être activés tant que les protections spécifiques à la plateforme ne sont pas terminé
es. Le code actuel ne garantit pas la création d'un stop-loss lié à l'ordre : garder le mode réel dés
activé.

Backtest : données requises
CSV avec en-têtes `timestamp,open,high,low,close,volume`; minimum 100 bougies, timestamps UTC croissants,
sans doublons, trous temporels anormaux, valeurs OHLC incohérentes, prix non positifs ni volumes nég
atifs. Le rapport produit des sections train/validation/test chronologiques et des variations de para
mètres prédéfinies. Les coûts sont des hypothèses que l'utilisateur doit calibrer au marché et à la p
lateforme.
La stratégie SMA/ATR intégrée est un exemple de pipeline de validation, pas une stratégie recommandée. Au
cun verdict « stratégie rentable » ne doit être produit automatiquement. Exiger des données provenant
de sources vérifiables, des splits chronologiques répétés, walk-forward, frais réalistes, test de se
nsibilité, paper trading prolongé et revue indépendante.

Tests
```bash
pytest -q
```
Les tests fournis couvrent des contrôles critiques de données et de limites, mais ne sont pas exhaustifs.
La production nécessite davantage de tests d'intégration avec la sandbox de la plateforme choisie et
des tests de sécurité indépendants.

Connecteurs ajoutés : Forex MT5 + crypto

Forex — MetaTrader 5
- Module `mt5_adapter.py`, exécuté sur **Windows** avec le terminal MetaTrader 5 installé et connecté à u
  n compte de démonstration.
- Envoie un ordre marché avec `sl` et `tp` inclus dans la requête native MT5; effectue `order_check`, con
  trôle le volume, puis vérifie les positions retournées par `positions_get`.
- Le compte réel est refusé par défaut. Pour un compte non démo, plusieurs drapeaux explicites sont requi
  s; ils ne doivent pas être activés avant revue humaine documentée.
- Le résultat d'un ordre ambigu n'est jamais rejoué automatiquement. En cas d'écart de rapprochement ou d
  e SL manquant, l'adaptateur lève une erreur urgente.
- Un compte de démonstration MT5 n'est pas une sandbox identique à tous les courtiers : les règles de vol
  ume, de remplissage, de stop level, de couverture/netting et de disponibilité doivent être testées ch
  ez le courtier cible.

Cryptomonnaies — CCXT
- Module `crypto_adapter.py`, configuré indépendamment de MT5 via `CRYPTO_EXCHANGE_ID`, `CRYPTO_API_KEY`
  et `CRYPTO_API_SECRET`.
- Sandbox/testnet activé par défaut; le code refuse d'envoyer une entrée si la plateforme ne déclare pas
  le support des SL/TP attachés pour le marché demandé.
- La prise en charge réelle dépend de l'exchange, du marché spot/dérivés et de la version CCXT. Les tests

doivent être exécutés sur le testnet de l'exchange choisi. Les comptes spot n'ont pas de notion univ
```

```
    erselle de « position » : rapprochement par soldes et ordres, tandis que les dérivés peuvent exposer
    `fetch_positions`.
- Aucun retry automatique après timeout : le résultat peut être ambigu. Il faut rapprocher ordres, trades
    , soldes et protections avant toute action.

### Routage unifié
`execution_router.py` expose `submit(venue="mt5"|"crypto", ...)` et `reconcile(...)`. Cela ne signifie pa
    s que les capitaux des deux comptes sont fusionnés : les soldes et risques doivent rester isolés par
    courtier et devise, puis être convertis explicitement pour un tableau consolidé.

### Blocage volontaire de l'API de trading réel
L'endpoint `/api/live/order` reste bloqué par HTTP 503. Les adaptateurs sont disponibles pour tests contr
    ôlés, mais l'application web n'a pas encore un gestionnaire de risque relié de façon fiable aux fonds
     propres, marges, positions et pertes réellement constatés sur chaque courtier, ni un cycle complet d
    e gestion des ordres partiellement exécutés, des annulations, des stops modifiés et des reconnexions.
     Ne pas supprimer ce blocage avant d'avoir implémenté ces contrôles, testé sur démo/testnet et fait r
    éaliser un audit indépendant.

### Tests et portée des résultats
Les tests `tests/test_adapters.py` utilisent des faux courtiers : ils testent les contrats et les garde-f
    ous, pas les serveurs de MT5 ni d'exchange. Ils ne constituent donc pas un test sandbox réel. L'envir
    onnement de génération peut ne pas contenir `MetaTrader5`, `ccxt` ou toutes les dépendances; installe
    r `requirements.txt` dans un environnement dédié avant de lancer `PYTHONPATH=. pytest -q`.

Sources techniques officielles consultées : [MetaTrader 5 Python — order_send](https://www.mql5.com/en/do
    cs/python_metatrader5/mt5ordersend_py), [positions_get](https://www.mql5.com/en/docs/python_metatrade
    r5/mt5positionsget_py), [manuel CCXT — ordres stop et take-profit attachés](https://docs.ccxt.com/fr/
    docs/manual).


## Connecteurs MT5 Forex et cryptomonnaies

Voir `INTEGRATION_MT5_CRYPTO.md` et `docs/Carnet_Trading_Addendum_MT5_Crypto.pdf`.
- Forex : adaptateur MetaTrader 5 (`mt5_adapter.py`), terminal Windows requis.
- Crypto : adaptateur CCXT (`crypto_adapter.py`), sandbox par défaut.
- Routage unifié : `execution_router.py`.
- Les ordres réels restent bloqués dans l'API web par défaut ; adaptateurs non audités.
- Les tests locaux sont simulés et ne remplacent pas les tests du compte démo MT5 ou du testnet de l'exch
    ange.

L'application n'est ni auditée indépendamment, ni certifiée, ni déclarée prête pour la production.
```

Annexe B — INTEGRATION_MT5_CRYPTO.md (texte intégral)

Fichier : `INTEGRATION_MT5_CRYPTO.md`

Addendum d'implémentation MT5 Forex et Crypto

```
# Addendum d'implémentation — Forex MT5 et cryptomonnaies

**Version de l'addendum : 1.3 — 23 octobre 2026**

## 1. Décision d'architecture

Le compte applicatif Carnet de Trading peut suivre deux connecteurs distincts :

1. **Forex / MetaTrader 5** — pont Python `MetaTrader5`, terminal MT5 installé sur Windows, compte démo r
    ecommandé pour l'intégration.
2. **Cryptomonnaies / CCXT** — connecteur crypto indépendant, plateforme choisie par configuration, testn
    et activé par défaut.

Le fait de les afficher dans une même application ne fusionne pas les soldes ni les risques. Chaque compt
    e garde sa propre devise, marge, exposition, frais et règles d'exécution. Une vue consolidée doit con
    vertir les valeurs au moyen d'un taux horodaté et garder les valeurs d'origine.

## 2. Modules livrés

- `mt5_adapter.py` : connexion, vérification compte/symbole/volume, `order_check`, envoi avec SL et TP na
    tifs inclus, rapprochement avec `positions_get`, refus d'un résultat non rapproché.
- `crypto_adapter.py` : CCXT, sandbox par défaut, vérification explicite des fonctionnalités `stopLoss` e
    t `takeProfit`, envoi d'une seule entrée avec protections attachées, aucun retry automatique après ré
    sultat ambigu, lecture soldes/ordres/positions si la plateforme les expose.
- `execution_router.py` : API Python unifiée `submit(venue=...)` et `reconcile(venue=...)`.
- `tests/test_adapters.py` : tests simulés de refus, paramètres SL/TP, garde-fous et absence de retry ave
    ugle.

## 3. Règles MT5

Le connecteur inclut le stop-loss dans la requête `TRADE_ACTION_DEAL`, avec un take-profit optionnel. Il
    vérifie le symbole, le tick, les limites et pas de volume, les directions SL/TP, puis utilise `order_
    check` avant `order_send`. Après envoi, il lit les positions et refuse de déclarer l'ordre protégé si
     une position correspondante ou un SL côté courtier n'est pas visible.

Le comportement dépend encore du courtier : mode netting/hedging, types de remplissage, stop level, volum
    e minimum, glissement et codes de retour. Les règles exactes doivent être validées sur le compte démo
     du courtier cible. L'installation Python MT5 est prévue sur Windows avec le terminal approprié.

## 4. Règles crypto

Les plateformes n'implémentent pas toutes les protections de la même manière. Le connecteur refuse l'ordr
    e si CCXT ne confirme pas explicitement le support des SL/TP attachés pour le marché choisi. Il ne te
    nte pas de substituer une entrée non protégée. La compatibilité doit être testée pour l'exchange, le
    symbole, spot ou dérivés, et la version de CCXT.

Pour un compte spot, les actifs sont principalement rapprochés via soldes, ordres et trades; la notion de
     position n'est pas standardisée comme pour les dérivés. Pour les dérivés, `fetch_positions` est util
    isé seulement si l'exchange déclare cette capacité. La gestion des positions courtes, `reduceOnly`, h
    edge mode, marge isolée/croisée et OCO doit être vérifiée par plateforme avant extension.

## 5. Verrous de sécurité

Les deux connecteurs exigent plusieurs drapeaux explicites avant l'envoi. Les valeurs par défaut maintien
    nent le réel désactivé. Les clés API doivent être conservées côté serveur, jamais dans le navigateur

    ou le dépôt. Pour les clés crypto, désactiver les retraits et limiter les permissions au minimum.

Aucun envoi n'est automatiquement rejoué après timeout : le serveur distant peut avoir accepté l'ordre ma
    lgré l'absence de réponse. La procédure est alors : geler les nouveaux envois, consulter les ordres e
    t transactions chez le courtier, rapprocher les positions et protections, puis résoudre manuellement
    l'état ambigu.

## 6. Pourquoi l'API web reste bloquée

L'endpoint `/api/live/order` reste bloqué par HTTP 503. Les adaptateurs et tests constituent l'intégratio
    n technique initiale, mais le moteur web ne possède pas encore de rapprochement complet et persistant
     de l'équité réelle, de la marge, des positions, des fills partiels, des ordres annulés, des protecti
    ons modifiées, des pertes journalières et du kill switch pour chaque courtier. Autoriser les ordres v
    ia l'interface sans ces contrôles serait dangereux.

Avant déverrouillage : connecter le gestionnaire de risque à l'état courtier, imposer une liste d'instrum
    ents autorisés, gérer idempotence et états d'ordre, vérifier les SL/TP après chaque fill, gérer les r
    econnexions et le kill switch, effectuer des tests de démo/testnet prolongés et commander un audit in
    dépendant.

## 7. Tests et état

Les tests automatiques locaux utilisent des faux courtiers et ne communiquent pas avec un serveur MT5 ni
    avec un testnet crypto. Dans l'environnement de génération, **13 tests des adaptateurs et du moteur d
    e backtest ont passé**. Cela ne signifie pas que les tests complets de l'application passent : certai
    nes dépendances n'étaient pas installées dans cet environnement. Installer `requirements.txt` puis la
    ncer `PYTHONPATH=. pytest -q` dans un environnement propre.
```

```
## 8. Statut de conformité

Ce projet **n'est pas audité, certifié ni prêt pour le trading réel**. L'implémentation peut contenir des
     défauts, les ordres peuvent subir du slippage ou des gaps, et les stops ne garantissent pas le prix
    d'exécution. Aucun backtest ne garantit de gains. Une revue indépendante du code, de la sécurité, des
     données et des opérations est requise.

## 9. Références techniques

- MetaTrader 5 Python — `order_send`: https://www.mql5.com/en/docs/python_metatrader5/mt5ordersend_py
- MetaTrader 5 Python — `positions_get`: https://www.mql5.com/en/docs/python_metatrader5/mt5positionsget_
    py
- Manuel CCXT — ordres stop, take-profit et protections attachées : https://docs.ccxt.com/fr/docs/manual
```